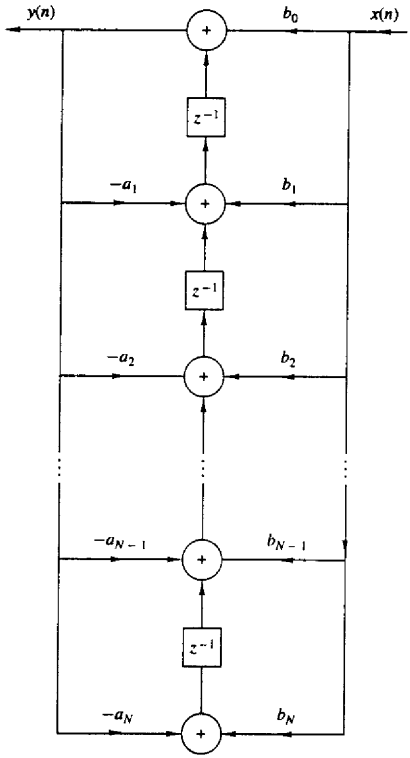


Figure 7.16 Transposed direct form II structure.

The transposed direct form II realization that we have obtained can be described by the set of difference equations

$$y(n) = w_1(n-1) + b_0 x(n) \tag{7.3.6}$$

$$w_k(n) = w_{k+1}(n-1) - a_k y(n) + b_k x(n) \qquad k = 1, 2, \ldots, N-1 \tag{7.3.7}$$

$$w_N(n) = b_N x(n) - a_N y(n) \tag{7.3.8}$$

Without loss of generality, we have assumed that $M = N$ in writing equations. It is also clear from observation of Fig. 7.17 that this set of difference equations is equivalent to the single difference equation

$$y(n) = -\sum_{k=1}^{N} a_k y(n-k) + \sum_{k=0}^{M} b_k x(n-k) \tag{7.3.9}$$

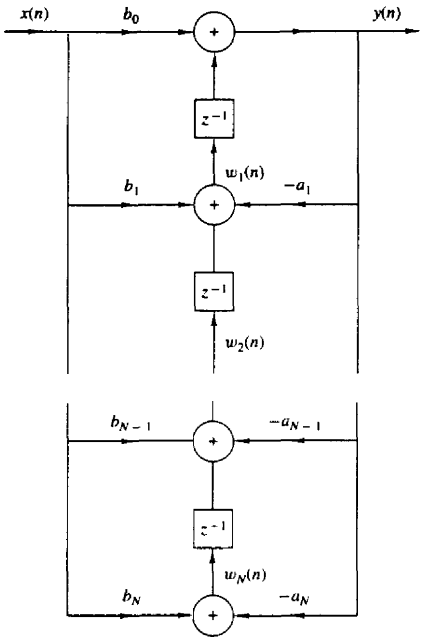


Figure 7.17 Transposed direct form II structure.

Finally, we observe that the transposed direct form II structure requires the same number of multiplications, additions, and memory locations as the original direct form II structure.

Although our discussion of transposed structures has been concerned with the general form of an IIR system, it is interesting to note that an FIR system, obtained from (7.3.9) by setting the $a_k = 0$, k = 1, 2, . . . , N, also has a transposed direct form as illustrated in Fig. 7.18. This structure is simply obtained from Fig. 7.17 by setting $a_k = 0$, k = 1, 2, . . . , N. This transposed form realization may

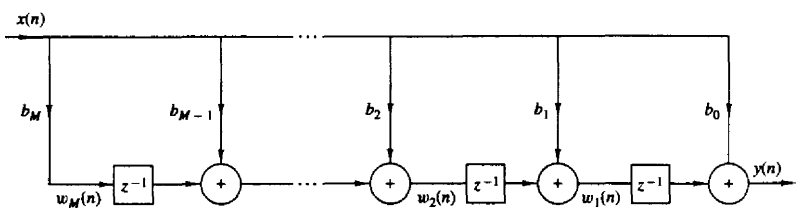


Figure 7.18 Transposed FIR structure.

be described by the set of difference equations

$$w_M(n) = b_M x(n) \tag{7.3.10}$$

$$w_k(n) = w_{k+1}(n-1) + b_k x(n) \qquad k = M-1, M-2, \ldots, 1 \tag{7.3.11}$$

$$y(n) = w_1(n-1) + b_0 x(n) \tag{7.3.12}$$

In summary, Table 7.1 illustrates the direct-form structures and the corresponding difference equations for a basic two-pole and two-zero IIR system with system function

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \tag{7.3.13}$$

This is the basic building block in the cascade realization of high-order IIR systems, as described in the following section. Of the three direct-form structures given in Table 7.1, the direct form II structures are preferable due to the smaller number of memory locations required in their implementation.

Finally, we note that in the z-domain, the set of difference equations describing a linear signal flow graph constitute a linear set of equations. Any rearrangement of such a set of equations is equivalent to a rearrangement of the signal flow graph to obtain a new structure, and vice versa.

7.3.3 Cascade-Form Structures

Let us consider a high-order IIR system with system function given by (7.1.2). Without loss of generality we assume that $N \geq M$. The system can be factored into a cascade of second-order subsystems, such that $H(z)$ can be expressed as

$$H(z) = \prod_{k=1}^{K} H_k(z) \tag{7.3.14}$$

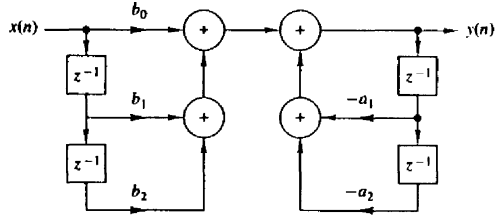
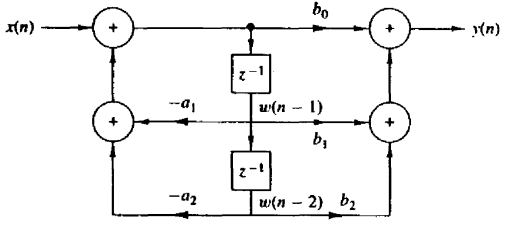
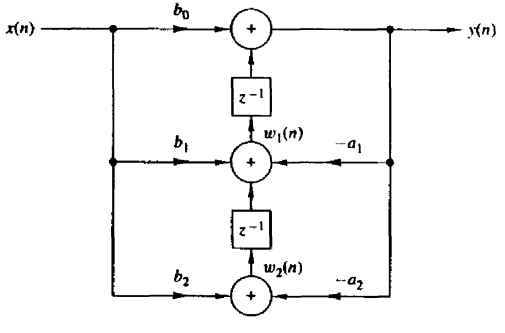
where K is the integer part of $(N+1)/2$. $H_k(z)$ has the genera1 form

$$H_k(z) = \frac{b_{k0} + b_{k1} z^{-1} + b_{k2} z^{-2}}{1 + a_{k1} z^{-1} + a_{k2} z^{-2}} \tag{7.3.15}$$

As in the case of FIR systems based on a cascade-form realization, the parameter b_0 can be distributed equally among the K filter sections so that $b_0 = b_{10} b_{20} \ldots b_{K0}$.

The coefficients $\{a_{ki}\}$ and $\{b_{ki}\}$ in the second-order subsystems are real. This implies that in forming the second-order subsystems or quadratic factors in (7.3.15), we should group together a pair of complex-conjugate poles and we should group together a pair of complex-conjugate zeros. However, the pairing of two complex-conjugate poles with a pair of complex-conjugate zeros or real-valued zeros to form a subsystem of the type given by (7.3.15), can be done arbitrarily. Furthermore, any two real-valued zeros can be paired together to form a quadratic factor and, likewise, any two real-valued poles can be paired together to form a quadratic factor. Consequently, the quadratic factor in the numerator of (7.3.15) may consist

TABLE 7.1 SOME *SECOND-ORDER* MODULES FOR DISCRETE-TIME SYSTEMS

	Structure	Implementation Equations	System Function
Direct Form I		$y(n) = b_0x(n) + b_1x(n-1) + b_2x(n-2) - a_1y(n-1) - a_2y(n-2)$	$H(z) = \dfrac{b_0 + b_1z^{-1} + b_2z^{-2}}{1 + a_1z^{-1} + a_2z^{-2}}$
Regular Direct Form II		$w(n) = -a_1w(n-1) - a_2w(n-2) + x(n)$ $y(n) = b_0w(n) + b_1w(n-1) + b_2w(n-2)$	$H(z) = \dfrac{b_0 + b_1z^{-1} + b_2z^{-2}}{1 + a_1z^{-1} + a_2z^{-2}}$
Transposed Direct Form II		$y(n) = b_0x(n) + w_1(n-1)$ $w_1(n) = b_1x(n) - a_1y(n) + w_2(n-1)$ $w_2(n) = b_2x(n) - a_2y(n)$	$H(z) = \dfrac{b_0 + b_1z^{-1} + b_2z^{-2}}{1 + a_1z^{-1} + a_2z^{-2}}$

of either a pair of real roots or a pair of complex-conjugate roots. The same statement applies to the denominator of (7.3.15).

If $N > M$, some of the second-order subsystems have numerator coefficients that are zero, that is, either $b_{k2} = 0$ or $b_{k1} = 0$ or both $b_{k2} = b_{k1} = 0$ for some k. Furthermore, if N is odd, one of the subsystems, say $H_k(z)$, must have $a_{k2} = 0$, so that the subsystem is of first order. To preserve the modularity in the implementation of $H(z)$, it is often preferable to use the basic second-order subsystems in the cascade structure and have some zero-valued coefficients in some of the subsystems.

Each of the second-order subsystems with system function of the form (7.3.15) can be realized in either direct form I, or direct form II, or transposed direct form II. Since there are many ways to pair the poles and zeros of H(z) into a cascade of second-order sections, and several ways to order the resulting subsystems, it is possible to obtain a variety of cascade realizations. Although all cascade realizations are equivalent for infinite precision arithmetic, the various realizations may differ significantly when implemented with finite-precision arithmetic.

The general form of the cascade structure is illustrated in Fig. 7.19. If we use the direct form II structure for each of the subsystems, the computational algorithm for realizing the IIR system with system function $H(z)$ is described by the following set of equations.

$$y_0(n) = x(n) \tag{7.3.16}$$

$$w_k(n) = -a_{k1}w_k(n-1) - a_{k2}w_k(n-2) + y_{k-1}(n) \qquad k = 1, 2, \ldots, K \tag{7.3.17}$$

$$y_k(n) = b_{k0}w_k(n) + b_{k1}w_k(n-1) + b_{k2}w_k(n-2) \qquad k = 1, 2, \ldots, K \tag{7.3.18}$$

$$y(n) = y_K(n) \tag{7.3.19}$$

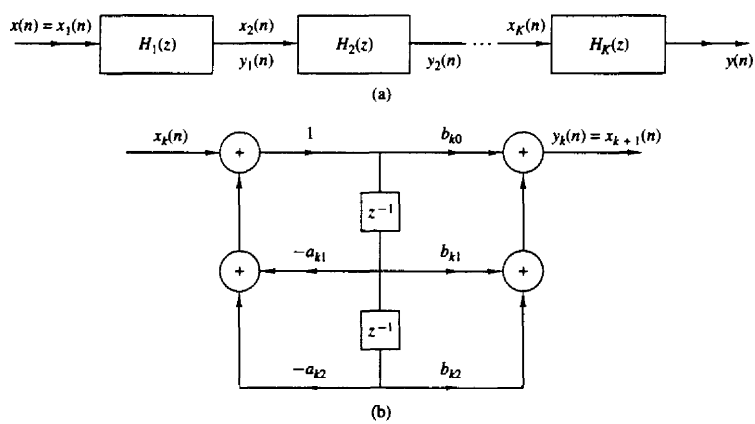


Figure 7.19 Cascade structure of second-order systems and a realization of each second-order section.

Thus this set of equations provides a complete description of the cascade structure based on direct form II sections.

7.3.4 Parallel-Form Structures

A parallel-form realization of an IIR system can be obtained by performing a partial-fraction expansion of $H(z)$. Without loss of generality, we again assume that N $\geq$ M and that the poles are distinct. Then, by performing a partial-fraction expansion of $H(z)$, we obtain the result

$$H(z) = C + \sum_{k=1}^{N} \frac{A_k}{1 - p_k z^{-1}} \tag{7.3.20}$$

where $\{p_k\}$ are the poles, $\{A_k\}$ are the coefficients (residues) in the partial-fraction expansion, and the constant C is defined as $C = b_N/a_N$. The structure implied by (7.3.20) is shown in Fig. 7.20. It consists of a parallel bank of single-pole filters.

In general, some of the poles of $H(z)$ may be complex valued. In such a case. the corresponding coefficients A_k are also complex valued. To avoid multiplications by complex numbers. we can combine pairs of complex-conjugate poles to form two-pole subsystems. In addition, we can combine. in an arbitrary manner,

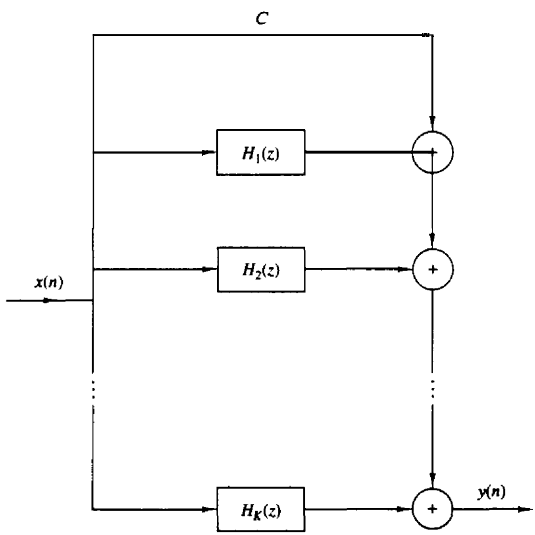


Figure 7.20 Parallel structure of IIR system.

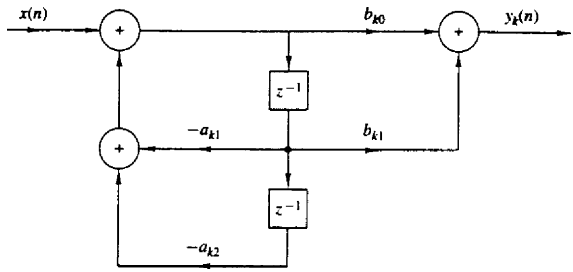


Figure 7.21 Structure of second-order section in a parallel IIR system realization.

pairs of real-valued poles to form two-pole subsystems. Each of these subsystems has the form

$$H_k(z) = \frac{b_{k0} + b_{k1}z^{-1}}{1 + a_{k1}z^{-1} + a_{k2}z^{-2}} \tag{7.3.21}$$

where the coefficients $\{b_{ki}\}$ and $\{a_{ki}\}$ are real-valued system parameters. The overall function can now be expressed as

$$H(z) = C + \sum_{k=1}^{K} H_k(z) \tag{7.3.22}$$

where K is the integer part of $(N+1)/2$. When N is odd, one of the $H_k(z)$ is really a single-pole system (i.e., $b_{k1} = a_{k2} = 0$).

The individual second-order sections which are the basic building blocks for $H(z)$ can be implemented in either of the direct forms or in a transposed direct form. The direct form II structure is illustrated in Fig. 7.21. With this structure as a basic building block, the parallel-form realization of the FIR system is described by the following set of equations

$$w_k(n) = -a_{k1}w_k(n-1) - a_{k2}w_k(n-2) + x(n) \qquad k = 1, 2, \ldots, K \tag{7.3.23}$$

$$y_k(n) = b_{k0}w_k(n) + b_{k1}w_k(n-1) \qquad k = 1, 2, \ldots, K \tag{7.3.24}$$

$$y(n) = Cx(n) + \sum_{k=1}^{K} y_k(n) \tag{7.3.25}$$

Example 7.3.1

Determine the cascade and parallel realizations for the system described by the system function

$$H(z) = \frac{10(1 - \frac{1}{2}z^{-1})(1 - \frac{2}{3}z^{-1})(1 + 2z^{-1})}{(1 - \frac{3}{4}z^{-1})(1 - \frac{1}{8}z^{-1})[1 - (\frac{1}{2} + j\frac{1}{2})z^{-1}][1 - (\frac{1}{2} - j\frac{1}{2})z^{-1}]}$$

Solution The cascade realization is easily obtained from this form. One possible pairing of poles and zeros is

$$H_1(z) = \frac{1 - \frac{2}{3}z^{-1}}{1 - \frac{7}{8}z^{-1} + \frac{3}{32}z^{-2}}$$

$$H_2(z) = \frac{1 + \frac{3}{2}z^{-1} - z^{-2}}{1 - z^{-1} + \frac{1}{2}z^{-2}}$$

and hence

$$H(z) = 10H_1(z)H_2(z)$$

The cascade realization is depicted in Fig. 7.22a.

To obtain the parallel-form realization, $H(z)$ must be expanded in partial fractions. Thus we have

$$H(z) = \frac{A_1}{1 - \frac{3}{4}z^{-1}} + \frac{A_2}{1 - \frac{1}{8}z^{-1}} + \frac{A_3}{1 - (\frac{1}{2} + j\frac{1}{2})z^{-1}} + \frac{A_3^*}{1 - (\frac{1}{2} - j\frac{1}{2})z^{-1}}$$

where A_1, A_2, A_3, and A_3^* are to be determined. After some arithmetic we find that

$$A_1 = 2.93, \quad A_2 = -17.68, \quad A_3 = 12.25 - \text{j}14.57. \quad A_3^* = 12.25 + \text{j}14.57$$

upon recombining pairs of poles. we obtain

$$H(z) = \frac{-14.75 - 12.90z^{-1}}{1 - \frac{7}{8}z^{-1} + \frac{3}{32}z^{-2}} + \frac{24.50 + 26.82z^{-1}}{1 - z^{-1} + \frac{1}{2}z^{-2}}$$

The parallel-form realization is illustrated in Fig. 7.22b.

7.3.5 Lattice and Lattice-Ladder Structures for IIR Systems

In Section 7.2.4 we developed a lattice filter structure that is equivalent to an FIR system. In this section we extend the development to IIR systems.

Let us begin with an all-pole system with system function

$$H(z) = \frac{1}{1 + \sum_{k=1}^{N} a_N(k)z^{-k}} = \frac{1}{A_N(z)} \tag{7.3.26}$$

The direct form realization of this system is illustrated in Fig. 7.23. The difference equation for this IIR system is

$$y(n) = -\sum_{k=1}^{N} a_N(k)y(n-k) + x(n) \tag{7.3.27}$$

It is interesting to note that if we interchange the roles of input and output [i.e., interchange $x(n)$ with $y(n)$ in (7.3.27)], we obtain

$$x(n) = -\sum_{k=1}^{N} a_N(k)x(n-k) + y(n)$$

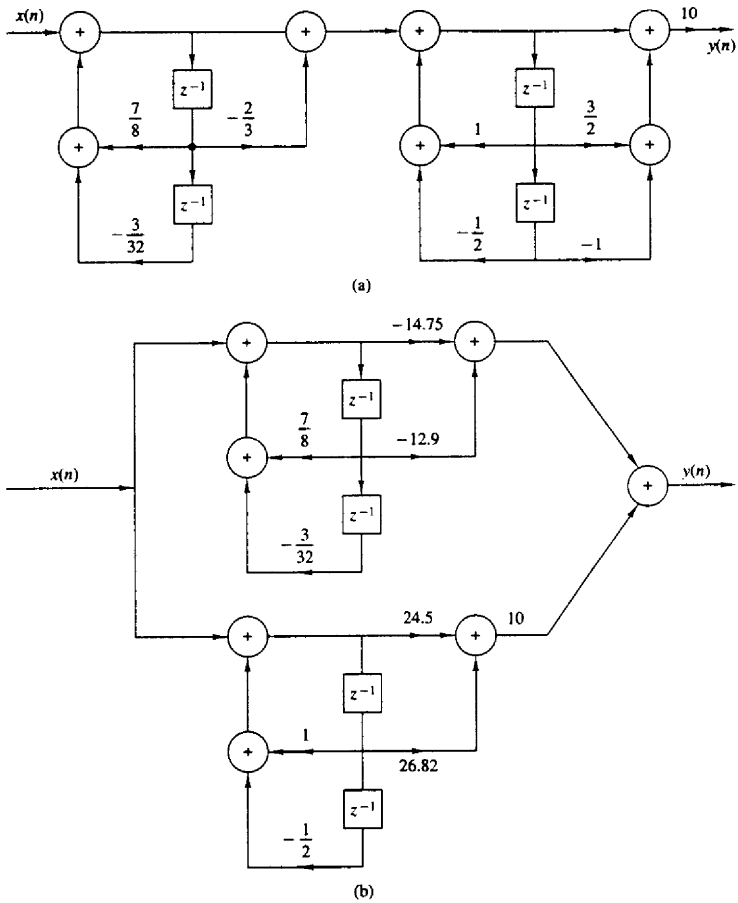


Figure 7.22 Cascade and parallel realizations for the system in Example 7.3.1.

or, equivalently,

$$y(n) = x(n) + \sum_{k=1}^{N} a_N(k)x(n-k) \tag{7.3.28}$$

We note that the equation in (7.3.28) describes an FIR system having the system function $H(z) = A_N(z)$, while the system described by the difference equation in (7.3.27) represents an IIR system with system function $H(z) = 1/A_N(z)$.

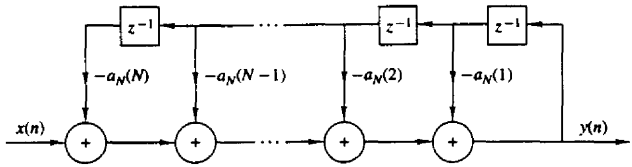


Figure 7.23 Direct-form realization of an all-pole system.

One system can be obtained from the other simply by interchanging the roles of the input and output.

Based on this observation, we shall use the all-zero (FIR) lattice described in Section *7.2.4* to obtain a lattice structure for an all-pole IIR system by interchanging the roles of the input and output. First, we take the all-zero lattice filter illustrated in Fig. 7.11 and then redefine the input as

$$x(n) = f_N(n) \tag{7.3.29}$$

and the output as

$$y(n) = f_0(n) \tag{7.3.30}$$

These are exactly the opposite of the definitions for the all-zero lattice filter. These definitions dictate that the quantities $\{f_m(n)\}$ be computed in descending order [i.e., $f_N(n)$, $f_{N-1}(n), \ldots$]. This computation can be accomplished by rearranging the recursive equation in *(7.2.29)* and thus solving for $f_{m-1}(n)$ in terms of $f_m(n)$, that is,

$$f_{m-1}(n) = f_m(n) - K_m g_{m-1}(n-1) \qquad m = N, N-1, \ldots, 1$$

The equation *(7.2.30)* for $g_m(n)$ remains unchanged.

The result of these changes is the set of equations

$$f_N(n) = x(n) \tag{7.3.31}$$

$$f_{m-1}(n) = f_m(n) - K_m g_{m-1}(n-1) \qquad m = N, N-1, \ldots, 1 \tag{7.3.32}$$

$$g_m(n) = K_m f_{m-1}(n) + g_{m-1}(n-1) \qquad m = N, N-1, \ldots, 1 \tag{7.3.33}$$

$$y(n) = f_0(n) = g_0(n) \tag{7.3.34}$$

which correspond to the structure shown in Fig. *7.24*.

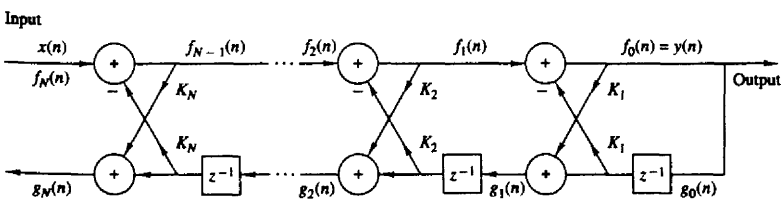


Figure 7.24 Lattice structure for an all-pole IIR system.

To demonstrate that the set of equations (7.3.31) through (7.3.34) represent an all-pole IIR system, let us consider the case where N = 1. The equations reduce to

$$\begin{aligned} x(n) &= f_1(n) \\ f_0(n) &= f_1(n) - K_1 g_0(n-1) \\ g_1(n) &= K_1 f_0(n) + g_0(n-1) \\ y(n) &= f_0(n) \\ &= x(n) - K_1 y(n-1) \end{aligned} \tag{7.3.35}$$

Furthermore, the equation for $g_1(n)$ can be expressed as

$$g_1(n) = K_1 y(n) + y(n-1) \tag{7.3.36}$$

We observe that (7.3.35) represents a first-order all-pole IIR system while (7.3.36) represents a first-order FIR system. The pote is a result of the feedback introduced by the solution of the $\{f_m(n)\}$ in descending order. This feedback is depicted in Fig. 7.25a.

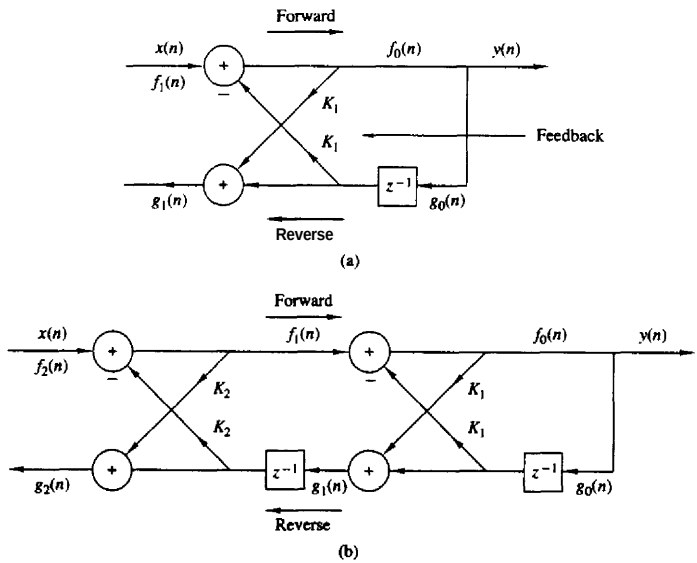


Figure 7.25 Singe-pole and two-pole lattice system.

Next, let us consider the case $N = 2$, which corresponds to the structure in Fig. 7.25b. The equations corresponding to this structure are

$$\begin{aligned} f_2(n) &= x(n) \\ f_1(n) &= f_2(n) - K_2 g_1(n-1) \\ g_2(n) &= K_2 f_1(n) + g_1(n-1) \\ f_0(n) &= f_1(n) - K_1 g_0(n-1) \\ g_1(n) &= K_1 f_0(n) + g_0(n-1) \\ y(n) &= f_0(n) = g_0(n) \end{aligned} \tag{7.3.37}$$

After some simple substitutions and manipulations 'we obtain

$$y(n) = -K_1(1 + K_2)y(n-1) - K_2 y(n-2) + x(n) \tag{7.3.38}$$

$$g_2(n) = K_2 y(n) + K_1(1 + K_2)y(n-1) + y(n-2) \tag{7.3.39}$$

Clearly. the difference equation in (7.3.38) represents a two-pole IJR system, and the relation in **(7.3.39)** is the input–output equation for a two-zero FIR system. Note that the coefficients for the FIR system are identical to those in the IIR system except that they occur in reverse order.

In general, these conclusions hold for any N. Indeed. with the definition of $A_m(z)$ given in **(7.2.32).** the system function for the all-pole IIR system is

$$H_a(z) = \frac{Y(z)}{X(z)} = \frac{F_0(z)}{F_m(z)} = \frac{1}{A_m(z)} \tag{7.3.40}$$

Similarly, the system function of the all-zero (FIR) system is

$$H_b(z) = \frac{G_m(z)}{Y(z)} = \frac{G_m(z)}{G_0(z)} = B_m(z) = z^{-m} A_m(z^{-1}) \tag{7.3.41}$$

where we used the previously established relationships in (7.2.36) through **(7.2.42).** Thus the coefficients in the **FIR** system $H_b(z)$ are identical to the coefficients in $A_m(z)$, except that they occur in reverse order.

It is interesting to note that the **all-pole** lattice structure has an all-zero path with input $g_0(n)$ and output $g_N(n)$, which is identical to its counterpart all-zero path in the all-zero lattice structure. The polynomial $B_m(z)$, which represents the system function of the all-zero path common to both lattice structures, is usually called the *backward* ***system*** *function*, because it provides a backward path in the all-pole lattice structure.

From this discussion the reader should observe that the all-zero and all-pole lattice structures are characterized by the same set of lattice parameters, namely, $K_1, K_2, \ldots, K_N$. The two lattice structures differ only in the interconnections of their signal flow graphs. Consequently, the algorithms for converting between the system parameters $\{\alpha_m(k)\}$ in the direct form realization of an **FIR** system, and the parameters of its lattice counterpart apply as well to the all-pole structure.

We recall that the roots of the polynomial $A_N(z)$ lie inside the unit circle if and only if the lattice parameters $|K_m| < 1$ for all $m = 1, 2, \ldots, N$. Therefore, the all-pole lattice structure is a stable system if and only if its parameters $|K_m| < 1$ for all m.

In practical applications the all-pole lattice structure has been used to model the human vocal tract and a stratified earth. In such cases the lattice parameters, $\{K_m\}$ have the physical significance of being identical to reflection coefficients in the physical medium. This is the reason that the lattice parameters are often called reflection *coefficients*. In such applications, a stable model of the medium requires that the reflection coefficients, obtained by performing measurements on output signals from the medium, be less than unity.

The all-pole lattice provides the basic building block for lattice-type structures that implement IIR systems that contain both poles and zeros. To develop the appropriate structure, let us consider an IIR system with system function

$$H(z) = \frac{\sum_{k=0}^{M} c_M(k) z^{-k}}{1 + \sum_{k=1}^{N} a_N(k) z^{-k}} = \frac{C_M(z)}{A_N(z)} \tag{7.3.42}$$

where the notation for the numerator polynomial has been changed to avoid confusion with our previous development. Without loss of generality, we assume that $N \geq M$.

In the direct form II structure, the system in (7.3.42) is described by the difference equations

$$w(n) = -\sum_{k=1}^{N} a_N(k) w(n-k) + x(n) \tag{7.3.43}$$

$$y(n) = \sum_{k=0}^{M} c_M(k) w(n-k) \tag{7.3.44}$$

Note that (7.3.43) is the input-output of an all-pole IIR system and that (7.3.44) is the input-output of an all-zero system. Furthermore, we observe that the output of the all-zero system is simply a linear combination of delayed outputs from the all-pole system. This is easily seen by observing the direct form II structure redrawn as in Fig. 7.26.

Since zeros result from forming a linear combination of previous outputs we can carry over this observation to construct a pole–zero IIR system using the all-pole lattice structure as the basic building block. We have already observed that $g_m(n)$ is a linear combination of present and past outputs. In fact, the system

$$H_b(z) = \frac{G_m(z)}{Y(z)} = B_m(z)$$

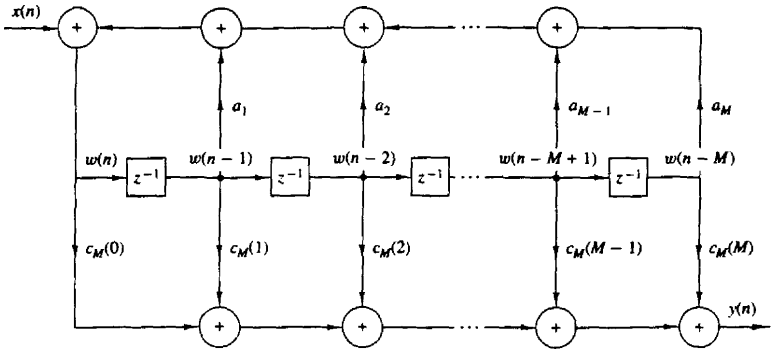


Figure 7.26 Direct form II realization of IIR system.

is an all-zero system. Therefore, any linear combination of $\{g_m(n)\}$ is also an all-zero system.

Thus we begin with an all-pole lattice structure with parameters K_m, $1 \leq m \leq N$, and we add a ladder part by taking as the output a weighted linear combination of $\{g_m(n)\}$. The result is a pole–zero IIR system which has the lattice-ladder structure shown in Fig. 7.27 for M = N. Its output is

$$y(n) = \sum_{m=0}^{M} v_m g_m(n) \tag{7.3.45}$$

where $\{v_m\}$ are the parameters that determine the zeros of the system. The system

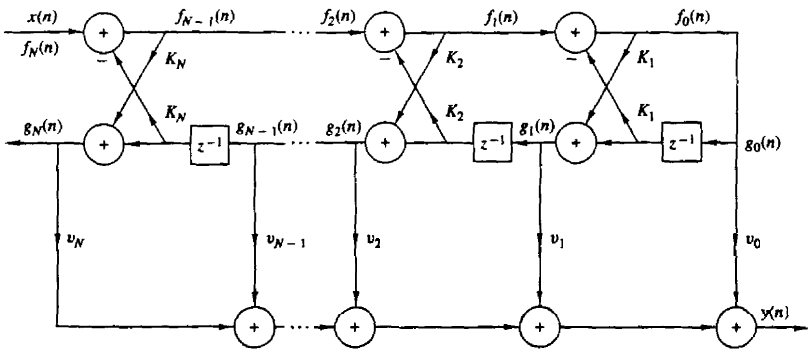


Figure 7.27 Lattice-ladder structure for the realization of a pole–zero system.

function corresponding to (7.3.45) is

$$H(z) = \frac{Y(z)}{X(z)} = \sum_{m=0}^{M} v_m \frac{G_m(z)}{X(z)} \tag{7.3.46}$$

Since $X(z) = F_N(z)$ and $F_0(z) = G_0(z)$, (7.3.46) can be written as

$$H(z) = \sum_{m=0}^{M} v_m \frac{G_m(z)}{G_0(z)} \frac{F_0(z)}{F_N(z)} = \sum_{m=0}^{M} v_m \frac{B_m(z)}{A_N(z)} = \frac{\sum_{m=0}^{M} v_m B_m(z)}{A_N(z)} \tag{7.3.47}$$

If we compare (7.3.41) with (7.3.47), we conclude that

$$C_M(z) = \sum_{m=0}^{M} v_m B_m(z) \tag{7.3.48}$$

This is the desired relationship that can be used to determine the weighting coefficients $\{v_m\}$. Thus, we have demonstrated that the coefficients of the numerator polynomial $C_M(z)$ determine the ladder parameters $\{v_m\}$, whereas the coefficients in the denominator polynomial $A_N(z)$ determine the lattice parameters $\{K_m\}$.

Given the polynomials $C_M(z)$ and $A_N(z)$, where $N \geq M$, the parameters of the all-pole lattice are determined first, as described previously. by the conversion algorithm given in Section 7.2.4, which converts the direct form coefficients into lattice parameters. By means of the step-down recursive relations given by (7.2.54), we obtain the lattice parameters $\{K_m\}$ and the polynomials $B_m(z)$, m = 1, 2, . . . , N.

The ladder parameters are determined from (7.3.48), which can be expressed as

$$C_m(z) = \sum_{k=0}^{m-1} v_k B_k(z) + v_m B_m(z) \tag{7.3.49}$$

or, equivalently, as

$$C_m(z) = C_{m-1}(z) + v_m B_m(z) \tag{7.3.50}$$

Thus $C_m(z)$ can be computed recursively from the reverse polynomials $B_m(z)$, m = 1, 2, . . . , M. Since $\beta_m(m) = 1$ for all m, the parameters v_m, $m = 0, 1, \ldots, M$ can be determined by first noting that

$$v_m = c_m(m) \qquad m = 0, 1, \ldots, M \tag{7.3.51}$$

Then, by rewriting (7.3.50) as

$$C_{m-1}(z) = C_m(z) - v_m B_m(z) \tag{7.3.52}$$

and running this recursive relation backward in m (i.e., $m = M, M-1, \ldots, 2$), we obtain $c_m(m)$ and therefore the ladder parameters according to (7.3.51).

The lattice-ladder filter structures that we have presented require the minimum amount of memory but not the minimum number of multiplications. Although lattice structures with only one multiplier per lattice stage exist, the two multiplier-per-stage lattice that we have described, is by far the most widely used in practical applications. In conclusion, the modularity, the built-in stability characteristics embodied in the coefficients $\{K_m\}$, and its robustness to finite-word-length effects make the lattice structure very attractive in many practical applications, including speech processing systems, adaptive filtering, and geophysical signal processing.

7.4 STATE-SPACE SYSTEM ANALYSIS AND STRUCTURES

Up to this point our treatment of linear time-invariant systems has been limited to an *input–output* or *external description* of the characteristics of the system. In other words, the system was characterized by mathematical equations that relate the input signal to the output signal. In this section we introduce the basic concepts in the state-space description of linear time-invariant causal systems. Although the *stare-space* or *internal description* of the system still involves a relationship between the input and output signals, it also involves an additional set of variables, called *state variables.* Furthermore, the mathematical equations describing the system, its input, and its output are usually divided into two parts:

1. A set of mathematical equations relating the state variables to the input signal.
2. A second set of mathematical equations relating the state variables and the current input to the output signal.

The state variables provide information about all the internal signals in the system. As a result, the state-space description provides a more detailed description of the system than the input–output description. Although our treatment of state-space analysis is confined primarily to single input–single output linear time-invariant causal systems, the state-space techniques can also be applied to nonlinear systems, time-variant systems, and multiple input–multiple output systems. In fact, it is in the characterization and analysis of multiple input–multiple output systems that the power and importance of state-space methods are clearly evident.

Both input–output and state-variable descriptions of a system are useful in practice. The description we use depends on the problem, the available information, and the questions to be answered. In our presentation, the emphasis is on

the use of state-space techniques in system analysis, and in the development of state-space structures for the realization of discrete-time systems.

7.4.1 State-Space Descriptions of Systems Characterized by Difference Equations

As we have already observed, the determination of the output of a system requires that we know the input signal and the set of initial conditions at the time the input is applied. If a system is not relaxed initially, say at time n_0, then knowledge of the input signal $x(n)$ for $n \geq n_0$ is not sufficient to uniquely determine the output $y(n)$ for $n \geq n_0$. The initial conditions of the system at $n = n_0$ must also be known and taken into account. This set of initial conditions is called the state of the system at $n = n_0$. Hence *we define the* state *of a system at time no as the amount of informarion that must be provided at time* n_0, *which, together with the input signal* $x(n)$ *for* $n \geq$ *no, uniquely determine the output of the system for all* $n \geq n_0$.

From this definition we infer that the concept of state leads to a decomposition of a system into two parts, a part that contains memory, and a memoryless component. The information stored in the memory component constitutes the set of initial conditions and is called the ***state of the system.*** The current output of the system then becomes a function of the current value of the input and the current state. Thus, to determine the output of the system at a given time, we need the current value of the state and the current input. Since the current value of the input is available, we only need to provide a mechanism for updating the state of the system recursively. Consequently, the state of the system at time $n_0 + 1$ should depend on the state of the system at time n_0 and the value of the input signal $x(n)$ at $n = no$.

The following example illustrates the approach in formulating a state-space description of a system. Let us consider a linear time-invariant causal system described by the difference equation

$$y(n) = -\sum_{k=1}^{3} a_k y(n-k) + \sum_{k=0}^{3} b_k x(n-k) \tag{7.4.1}$$

The direct form II realization for the system is shown in Fig. 7.28.

As state variables, we use the contents of the system memory registers, counting them from the bottom, as shown in Fig. 7.28. We recall that the output of a delay element represents the present value stored in the register and the input represents the next value to be stored in the memory. Consequently, with the aid of Fig. 7.28, we can write

$$\begin{aligned} v_1(n+1) &= v_2(n) \\ v_2(n+1) &= v_3(n) \\ v_3(n+1) &= -a_3 v_1(n) - a_2 v_2(n) - a_1 v_3(n) + x(n) \end{aligned} \tag{7.4.2}$$

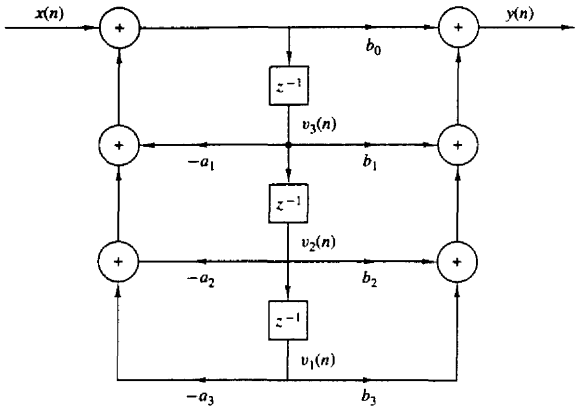


Figure 7.28 Direct form II realization of system described by the difference equation in (7.5.1).

It is interesting to note that the state-variable formulation for the third-order system of (7.4.1) involves three first-order difference equations given by (7.4.2). In general, an nth-order system can be described by n first-order difference equations.

The output equation, which expresses $y(n)$ in terms of the state variables and the present input value $x(n)$, can also be obtained by referring to Fig. 7.28. We have

$$y(n) = b_0 v_3(n+1) + b_3 v_1(n) + b_2 v_2(n) + b_1 v_3(n)$$

We can eliminate $v_3(n+1)$ by using the last equation in (7.4.2). Thus we obtain the desired output equation

$$y(n) = (b_3 - b_0 a_3) v_1(n) + (b_2 - b_0 a_2) v_2(n) + (b_1 - b_0 a_1) v_3(n) + b_0 x(n) \tag{7.4.3}$$

If we put (7.4.2) and (7.4.3) into matrix form we have

$$\begin{bmatrix} v_1(n+1) \\ v_2(n+1) \\ v_3(n+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -a_3 & -a_2 & -a_1 \end{bmatrix} \begin{bmatrix} v_1(n) \\ v_2(n) \\ v_3(n) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} x(n) \tag{7.4.4}$$

and

$$y(n) = [(b_3 - b_0 a_3) \; (b_2 - b_0 a_2) \; (b_1 - b_0 a_1)] \begin{bmatrix} v_1(n) \\ v_2(n) \\ v_3(n) \end{bmatrix} + b_0 x(n) \tag{7.4.5}$$

The equations (7.4.4) and (7.4.5) provide a complete description of the system. Furthermore, the variables $v_1(n)$, $v_2(n)$, and $v_3(n)$, which summarize all the necessary past information, are the ***state variables*** of the system. We also observe that as indicated previously, equations (7.4.4) and (7.4.5) split the system into two component parts, a dynamic (memory) subsystem and a static (memoryless) subsystem. We say that this set of equations provides a ***state-space description*** of the system.

By generalizing the previous example, it can easily be seen that the Nth-order system described by

$$y(n) = -\sum_{k=1}^{N} a_k y(n-k) + \sum_{k=0}^{N} b_k x(n-k) \tag{7.4.6}$$

can be expressed as a linear time-invariant state-space realization by the relations

State equation

$$\mathbf{v}(n+1) = \mathbf{F}\mathbf{v}(n) + \mathbf{q}x(n) \tag{7.4.7}$$

Output equation

$$y(n) = \mathbf{g}'\mathbf{v}(n) + dx(n) \tag{7.4.8}$$

where the elements of F, q, g, and d are constants (i.e., they do not change as a function of the time index n), given by

$$\mathbf{F} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & \\ & & & & \\ 0 & 0 & \cdots & 0 & 1 \\ -a_N & -a_{n-1} & \cdots & -a_2 & -a_1 \end{bmatrix} \qquad \mathbf{q} = \begin{bmatrix} \\ \\ \\ \\ \end{bmatrix} \tag{7.4.9}$$

$$\mathbf{g} = \begin{bmatrix} b_N - b_0 a_N \\ b_{N-1} - b_0 a_{N-1} \\ \vdots \\ b_1 - b_0 a_1 \end{bmatrix}$$

Any discrete-time system whose input $x(n)$, output $y(n)$, and state $\mathbf{v}(n)$, for all $n \geq n_0$, are related by the state-space equations above, where F, q, g, and d are arbitrary but fixed quantities, will be called linear and time invariant. If at least one of the quantities in F, q, g, or d depends on time, the system becomes time variant.

We will refer to (7.4.7) through (7.4.8) as the linear time-invariant state-space model, which can be represented by the simple vector-matrix block diagram in Fig. 7.29. In this figure the double lines represent vector quantities and the blocks represent the vector or matrix coefficients.

Example 7.4.1

Determine the state-space equations for the transposed direct form II structure shown in Fig. 7.30.

Solution The validity of this structure can be seen if we rewrite (7.4.1) as

$$y(n) = \sum_{k=1}^{3} [b_k x(n-k) - a_k y(n-k)] + b_0 x(n)$$

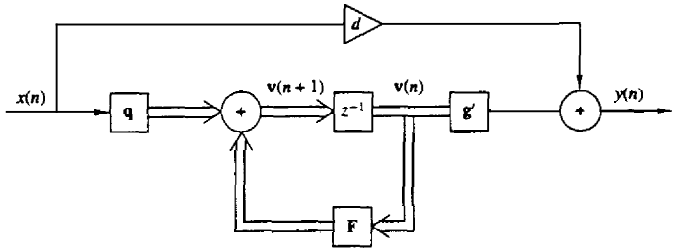


Figure 7.29 General state-space description of a linear time-invariant system.

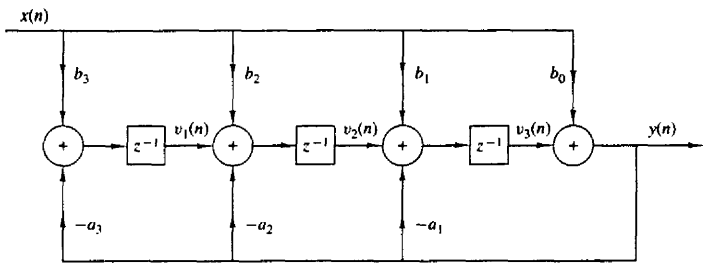


Figure 7.30 State-space realization for the system described by (7.4.1).

Due to the linearity and time invariance of the system, instead of first delaying the signals $x(n)$ and $y(n)$ and then computing the terms $b_k x(n-k) - a_k y(n-k)$ as in Fig. 7.28, we first compute the terms $b_k x(n) - a_k y(n)$ and then delay them.

If we use the state variables indicated in Fig. 7.30. we obtain

$$\begin{bmatrix} v_1(n+1) \\ v_2(n+1) \\ v_3(n+1) \end{bmatrix} = \begin{bmatrix} 0 & 0 & -a_3 \\ 1 & 0 & -a_2 \\ 0 & 1 & -a_1 \end{bmatrix} \begin{bmatrix} v_1(n) \\ v_2(n) \\ v_3(n) \end{bmatrix} + \begin{bmatrix} b_3 - b_0 a_3 \\ b_2 - b_0 a_2 \\ b_1 - b_0 a_1 \end{bmatrix} x(n) \tag{7.4.10}$$

$$y(n) = [0 \quad 0 \quad 1] \begin{bmatrix} v_1(n) \\ v_2(n) \\ v_3(n) \end{bmatrix} + b_0 x(n) \tag{7.4.11}$$

The state-space description specified by (7.4.4) and (7.4.5) is known as a type I state-space realization, whereas the one described by (7.4.10) and (7.4.11) is called a *type* 2 state-space realization.

7.4.2 Solution of the State-Space Equations

There are several methods for solving the state-space equations. Here we discuss a recursive solution which makes use of the fact that the state-space equations are a set of linear first-order difference equations.

For the N-dimensional state-space model

$$\mathbf{v}(n+1) = \mathbf{F}\mathbf{v}(n) + \mathbf{q}x(n) \tag{7.4.12}$$

$$y(n) = \mathbf{g}'\mathbf{v}(n) + dx(n) \tag{7.4.13}$$

and given the initial condition $\mathbf{v}(n_0)$, we have for $n > n_0$,

$$\begin{aligned}\mathbf{v}(n_0+1) &= \mathbf{F}\mathbf{v}(n_0) + \mathbf{q}x(n)\\ \mathbf{v}(n_0+2) &= \mathbf{F}\mathbf{v}(n_0+1) + \mathbf{q}x(n_0+1)\\ &= \mathbf{F}^2\mathbf{v}(n_0) + \mathbf{F}\mathbf{q}x(n_0) + \mathbf{q}x(n_0+1)\end{aligned}$$

where $\mathbf{F}^2$ represents the matrix product $\mathbf{FF}$ and $\mathbf{Fq}$ is the product of the matrix F and the vector $\mathbf{q}$. If we continue as in the one-dimensional case. we obtain, for $n > n_0$,

$$\mathbf{v}(n) = \mathbf{F}^{n-n_0}\mathbf{v}(n_0) + \sum_{k=n_0}^{n-1} \mathbf{F}^{n-1-k}\mathbf{q}x(k) \tag{7.4.14}$$

The matrix $\mathbf{F}^0$ is defined as the N $\times$ N identity matrix, having unity on the main diagonal and zeros elsewhere. The matrix $\mathbf{F}^{i-j}$ is often denoted as $\Phi(i-j)$, that is,

$$\Phi(i-j) = \mathbf{F}^{i-j} \tag{7.4.15}$$

for any positive integers $i \geq j$. This matrix is called the *state transition* matrix of the system.

The output of the system is obtained by substituting (7.4.14) into (7.4.13). The result of this substitution is

$$\begin{aligned}y(n) &= \mathbf{g}'\mathbf{F}^{n-n_0}\mathbf{v}(n_0) + \sum_{k=n_0}^{n-1} \mathbf{g}'\mathbf{F}^{n-1-k}\mathbf{q}x(k) + dx(n)\\ &= \mathbf{g}'\Phi(n-n_0)\mathbf{v}(n_0) + \sum_{k=n_0}^{n-1} \mathbf{g}'\Phi(n-1-k)\mathbf{q}x(k) + dx(n)\end{aligned} \tag{7.4.16}$$

From this general result, we can determine the output for two special cases. First, the zero-input response of the system is

$$y_{zi}(n) = \mathbf{g}'\mathbf{F}^{n-n_0}\mathbf{v}(n_0) = \mathbf{g}'\Phi(n-n_0)\mathbf{v}(n_0) \tag{7.4.17}$$

On the other hand, the zero-state response is

$$y_{zs}(n) = \sum_{k=n_0}^{n-1} \mathbf{g}'\Phi(n-1-k)\mathbf{q}x(k) + dx(n) \tag{7.4.18}$$

Clearly, the N-dimensional state-space system is zero-input linear, zero-state linear, and since $y(n) = y_{zi}(n) + y_{zs}(n)$, it is linear. Furthermore, since any system described by a linear constant-coefficient difference equation can be put in the state-space form, it is linear, in agreement with the results obtained in Section 2.4.

7.4.3 Relationships Between Input-Output and State-Space Descriptions

From our previous discussion we have seen that there is no unique choice for the state variables of a causal system. Furthermore, different choices for the state vector lead to different structures for the realization of the same system. Hence, in general, the input-output relationship does not uniquely describe the internal structure of the system.

To illustrate these assertions, let us consider an N-dimensional system with the state-space representation

$$\mathbf{v}(n+1) = \mathbf{F}\mathbf{v}(n) + \mathbf{q}x(n) \tag{7.4.19}$$

$$y(n) = \mathbf{g}'\mathbf{v}(n) + dx(n) \tag{7.4.20}$$

Let $\mathbf{P}$ be any N x N matrix whose inverse matrix $\mathbf{P}^{-1}$ exists. We define a new state vector $\hat{\mathbf{v}}(n)$ as

$$\hat{\mathbf{v}}(n) = \mathbf{P}\mathbf{v}(n) \tag{7.4.21}$$

Then

$$\mathbf{v}(n) = \mathbf{P}^{-1}\hat{\mathbf{v}}(n) \tag{7.4.22}$$

If (7.4.19) is premultiplied by $\mathbf{P}$, we obtain

$$\mathbf{P}\mathbf{v}(n+1) = \mathbf{P}\mathbf{F}\mathbf{v}(n) + \mathbf{P}\mathbf{q}x(n)$$

By using (7.4.22), the state equation above becomes

$$\hat{\mathbf{v}}(n+1) = (\mathbf{P}\mathbf{F}\mathbf{P}^{-1})\hat{\mathbf{v}}(n) + (\mathbf{P}\mathbf{q})x(n) \tag{7.4.23}$$

Similarly, with the aid of (7.4.22) the output equation (7.4.20) becomes

$$y(n) = (\mathbf{g}'\mathbf{P}^{-1})\hat{\mathbf{v}}(n) + dx(n) \tag{7.4.24}$$

Now, we define a new system parameter matrix $\hat{\mathbf{F}}$ and the vectors $\hat{\mathbf{q}}$ and $\hat{\mathbf{g}}$ as

$$\begin{aligned} \hat{\mathbf{F}} &= \mathbf{P}\mathbf{F}\mathbf{P}^{-1} \\ 4 &= \mathbf{P}\mathbf{q} \\ \hat{\mathbf{g}}' &= \mathbf{g}'\mathbf{P}^{-1} \end{aligned} \tag{7.4.25}$$

With these definitions, the state equations can be expressed in terms of the new system quantities as

$$\hat{\mathbf{v}}(n+1) = \hat{\mathbf{F}}\hat{\mathbf{v}}(n) + \hat{\mathbf{q}}x(n) \tag{7.4.26}$$

$$y(n) = \hat{\mathbf{g}}'\hat{\mathbf{v}}(n) + dx(n) \tag{7.4.27}$$

If we compare (7.4.19) and (7.4.20) with (7.4.26) and (7.4.27), we observe that by a simple linear transformation of the state variables, we have generated a new set of state equations and an output equation, in which the input $x(n)$ and the output $y(n)$ are unchanged. Since there is an infinite number of choices of the transformation matrix P, there is also an infinite number of state-space equations

and structures for a system. Some of these structures are different, while some others are very similar, differing only by scale factors.

Associated with any state-space realization of a system is the concept of a *minimal realization.* A state-space realization is said to be *minimal* if the dimension of the state space (the number of state variables) is the smallest of all possible realizations. Since each state variable represents a quantity that must be stored and updated at every time instant n, it follows that a minimal realization is one that requires the smallest number of delays (storage registers). We recall that the direct form II realization requires the smallest number of storages registers, and consequently, a state-space realization based on the contents of the delay elements results in a minimal realization. Similarly, an FIR system realized as a direct form structure leads to a minimal state-space realization if the values of the storage registers are defined as the state variables. On the other hand, the direct form I realization of an IIR system does not lead to a minimal realization.

Now, let us determine the impulse response of the system from the state-space realization. The impulse response provides one of the links between the input-output and state-space description of systems.

By definition the impulse response $h(n)$ of a system is the zero-state response of the system to the excitation $x(n) = \delta(n)$. Hence it can be obtained from equation (7.4.16) if we set $n_0 = 0$ (the time we apply the input), $\mathbf{v}(0) = 0$, and $x(n) = \delta(n)$. Thus the impulse response of the system described by (7.4.19) and (7.4.20) is given by

$$\begin{aligned} h(n) &= \mathbf{g}'\mathbf{F}^{n-1}\mathbf{q}u(n-1) + d\delta(n) \\ &= \mathbf{g}'\Phi(n-1)\mathbf{q}u(n-1) + d\delta(n) \end{aligned} \tag{7.4.28}$$

Given a state-space description, it is straightforward to determine the impulse response from (7.4.28). However, the inverse is not easy since there is an infinite number of state-space realizations for the same input-output description.

The transpose system. The transpose of a matrix F is obtained by interchanging its columns and rows, and it is denoted by $\mathbf{F}^t$. For example,

$$\mathbf{F} = \begin{bmatrix} f_{11} & f_{12} & \cdots & f_{1N} \\ f_{21} & f_{22} & \cdots & f_{2N} \\ \vdots & \vdots & \cdots & \vdots \\ f_{N1} & f_{N2} & \cdots & f_{NN} \end{bmatrix}, \qquad \mathbf{F}^t = \begin{bmatrix} f_{11} & f_{21} & \cdots & f_{N1} \\ f_{12} & f_{22} & \cdots & f_{N2} \\ \vdots & \vdots & \cdots & \vdots \\ f_{1N} & f_{2N} & \cdots & f_{NN} \end{bmatrix}$$

Now define the transpose system (7.4.19)–(7.4.20) as

$$\mathbf{v}'(n+1) = \mathbf{F}^t\mathbf{v}'(n) + \mathbf{g}x(n) \tag{7.4.29}$$

$$y'(n) = \mathbf{q}'\mathbf{v}'(n) + dx(n) \tag{7.4.30}$$

According to (7.4.28), the impulse response of this system is given as

$$h'(n) = \mathbf{q}'(\mathbf{F}^t)^{n-1}\mathbf{g}u(n-1) + d\delta(n) \tag{7.4.31}$$

From matrix algebra we know that $(\mathbf{F}')^{n-1} = (\mathbf{F}^{n-1})'$. Hence

$$h'(n) = \mathbf{q}'(\mathbf{F}^{n-1})'\mathbf{g}u(n-1) + d\delta(n)$$

We claim that $h'(n) = h(n)$. Indeed, the term $\mathbf{q}'(\mathbf{F}^{n-1})'\mathbf{g}$ is a scalar. Hence it is equal to its transpose. Consequently,

$$[\mathbf{q}'(\mathbf{F}^{n-1})'\mathbf{g}]' = \mathbf{g}'(\mathbf{F}')^{n-1}\mathbf{q}$$

Since this is true, it follows that (7.4.31) is identical to (7.4.28) and, therefore, $h'(n) = h(n)$. Thus a ***single input–single output system and its transpose have identical impulse responses and hence the same input-output relationship.*** To support this claim further, we note that the type 1 and type 2 state-space realizations described by (7.4.3), (7.4.4), (7.4.10), and (7.4.11) are transpose structures, which stem from the same input-output relationship (7.4.1).

We have introduced the transpose structure because it provides an easy method for generating a new structure. However, sometimes this new structure may either differ trivially or be identical to the original one.

The diagonal system. A closed-form solution of the state-space equations is easily obtained when the system matrix F is diagonal. Hence, by finding a matrix P so that $\hat{\mathbf{F}} = \mathbf{PFP}^{-1}$ is diagonal, the solution of the state equations is simplified considerably. The diagonalization of the matrix F can be accomplished by first determining the eigenvalues and eigenvectors of the matrix.

A number λ is an *eigenvalue* of F and a nonzero vector $\mathbf{u}$ is the associated *eigenvector* if

$$\mathbf{Fu} = \lambda\mathbf{u} \tag{7.4.32}$$

To determine the eigenvalues of F, we note that

$$(\mathbf{F} - \lambda\mathbf{I})\mathbf{u} = 0 \tag{7.4.33}$$

This equation has a (nontrivial) nonzero solution $\mathbf{u}$ if the matrix $\mathbf{F} - \lambda\mathbf{I}$ is singular [i.e., if $(\mathbf{F} - \lambda\mathbf{I})$ is noninvertible], which is the case if the determinant of $(\mathbf{F} - 11)$ is zero, that is, if

$$\det(\mathbf{F} - \lambda\mathbf{I}) = 0 \tag{7.4.34}$$

This determinant in (7.4.34) yields the ***characteristic polynomial*** of the matrix F. For an N x N matrix F, the characteristic polynomial of F is degree N and hence it has N roots, say λ_i, $i = 1, 2, \ldots, N$. The roots may be distinct or some roots may be repeated. In any case, for each root λ_i, we can determine a vector $\mathbf{u}_i$, called the eigenvector corresponding to the eigenvalue λ_i, from the equation

$$\mathbf{Fu}_i = \lambda_i\mathbf{u}_i$$

These eigenvectors are orthogonal, that is, $\mathbf{u}_i'\mathbf{u}_j = 0$, for $i \neq j$.

If we form a matrix U whose columns consist of the eigenvectors $\{\mathbf{u}_i\}$, that is,

$$\mathbf{U} = \begin{bmatrix} \uparrow & \uparrow & & \uparrow \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_N \\ \downarrow & \downarrow & & \downarrow \end{bmatrix}$$

then the matrix $\hat{\mathbf{F}} = \mathbf{U}^{-1}\mathbf{F}\mathbf{U}$ is diagonal. Thus we have solved for the matrix that diagonalizes F.

The following example illustrates the procedure of diagonalizing F.

Example 7.4.2

The Fibonacci sequence, which is the sequence $\{1, 1, 2, 3, 5, 8, 13, \ldots\}$, can be generated as the impulse response of the system that satisfies the state-space equations

$$\mathbf{v}(n+1) = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}\mathbf{v}(n) + \begin{bmatrix} 0 \\ 1 \end{bmatrix}x(n)$$

$$y(n) = \begin{bmatrix} 1 & 1 \end{bmatrix}\mathbf{v}(n) + x(n)$$

Determine the impulse response $\{h(n)\}$ of the system.

Solution Now we wish to determine an equivalent system

$$\hat{\mathbf{v}}(n+1) = \hat{\mathbf{F}}\hat{\mathbf{v}}(n) + \hat{\mathbf{q}}x(n)$$

$$y(n) = \hat{\mathbf{g}}'\hat{\mathbf{v}}(n) + dx(n)$$

such that the matrix $\hat{\mathbf{F}}$ is diagonal. From (7.4.25) we recall that the two systems are equivalent if

$$\hat{\mathbf{F}} = \mathbf{P}\mathbf{F}\mathbf{P}^{-1} \qquad \hat{\mathbf{q}} = \mathbf{P}q \qquad \hat{\mathbf{g}}' = \mathbf{g}'\mathbf{P}^{-1}$$

Given F, the problem is to determine a matrix P such that $\hat{\mathbf{F}} = \mathbf{P}\mathbf{F}\mathbf{P}^{-1}$ is a diagonal matrix.

First, we compute the determinant in (7.4.34). We have

$$\det(\mathbf{F} - \lambda\mathbf{I}) = \det\begin{bmatrix} -\lambda & 1 \\ 1 & 1-\lambda \end{bmatrix} = \lambda^2 - \lambda - 1 = 0$$

or

$$\lambda_1 = \frac{1+\sqrt{5}}{2} \qquad \lambda_2 = \frac{1-\sqrt{5}}{2}$$

To find the eigenvector $\mathbf{u}_1$ corresponding to λ_1, we have

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}\mathbf{u}_1 = \lambda_1\mathbf{u}_1 \qquad \text{or} \qquad \mathbf{u}_1 = \begin{bmatrix} 1 \\ \lambda_1 \end{bmatrix}$$

Similarly, we obtain

$$\mathbf{u}_2 = \begin{bmatrix} 1 \\ \lambda_2 \end{bmatrix}$$

We observe that $\mathbf{u}_1'\mathbf{u}_2 = 1 + \lambda_1\lambda_2 = 0$ (i.e., the eigenvectors are orthogonal). Now matrix U, whose columns are the eigenvectors of F, is

$$\mathbf{U} = \begin{bmatrix} 1 & 1 \\ \lambda_1 & \lambda_2 \end{bmatrix}$$

Then the matrix $\mathbf{U}^{-1}\mathbf{F}\mathbf{U}$ is diagonal. Indeed, it easily follows that

$$\hat{\mathbf{F}} = \mathbf{U}^{-1}\mathbf{F}\mathbf{U} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

and since the transformation matrix is $\mathbf{P} = \mathbf{U}^{-1}$, we have

$$\mathbf{P} = \frac{1}{\lambda_2 - \lambda_1}\begin{bmatrix} \lambda_2 & -1 \\ -\lambda_1 & 1 \end{bmatrix}$$

Thus the diagonal matrix $\hat{\mathbf{F}}$ has the form

$$\hat{\mathbf{F}} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

where the diagonal elements are the eigenvalues of the characteristic polynomial.

Furthermore, we obtain

$$\hat{\mathbf{q}} = \mathbf{P}\mathbf{q} = \begin{bmatrix} \frac{1}{\sqrt{5}} \\ -\frac{1}{\sqrt{5}} \end{bmatrix}$$

and

$$\hat{\mathbf{g}}' = \mathbf{g}'\mathbf{P}^{-1} = \mathbf{g}'\mathbf{U} = \begin{bmatrix} \frac{3+\sqrt{5}}{2} & \frac{3-\sqrt{5}}{2} \end{bmatrix}$$

The impulse response of this equivalent diagonal system is

$$h(n) = \hat{\mathbf{g}}'\hat{\mathbf{F}}\hat{\mathbf{q}}u(n-1) + d\delta(n)$$

$$= \frac{1}{\sqrt{5}}\left[\left(\frac{3+\sqrt{5}}{2}\right)\left(\frac{1+\sqrt{5}}{2}\right)^{n-1} - \left(\frac{3-\sqrt{5}}{2}\right)\left(\frac{1-\sqrt{5}}{2}\right)^{n-1}\right]u(n-1) + \delta(n)$$

which is the general formula for the Fibonacci sequence.

An alternative expression can be found by noting that the Fibonacci sequence can be considered as the zero-input response of the system described by the difference equation

$$y(n) = y(n-1) + y(n-2) + x(n)$$

with initial conditions $y(-1) = 1$, $y(-2) = -1$. From the type 1 state-space realization, we note that $v_1(0) = y(-2) = -1$ and $v_2(0) = y(-1) = 1$. Hence

$$\begin{bmatrix} \hat{v}_1(0) \\ \hat{v}_2(0) \end{bmatrix} = \mathbf{P}\begin{bmatrix} v_1(0) \\ v_2(0) \end{bmatrix} = \frac{-1}{5}\begin{bmatrix} \frac{-3+\sqrt{5}}{2} \\ \frac{3+\sqrt{5}}{2} \end{bmatrix}$$

and the zero-input response is

$$y_{zi}(n) = \hat{\mathbf{g}}'\hat{\mathbf{F}}^n\hat{\mathbf{v}}(0)$$

$$= \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]u(n)$$

This is the more familiar form for the Fibonacci sequence, where the first term of the sequence is zero, that is, the sequences is $\{0, 1, 1, 2, 3, 5, 8, \ldots\}$.

This example illustrates the method for diagonalizing the matrix F. The diagonal system yields a set of N decoupled, first-order linear difference equations that are easily solved to yield the state and the output of the system.

It is important to note that the eigenvalues of the matrix F are identical to the roots of the characteristic polynomial, which are obtained from the homogeneous difference equation that characterizes the system. For example. the system that generates the Fibonacci sequence is characterized by the homogeneous difference equation

$$y(n) - y(n-1) - y(n-2) = 0 \tag{7.4.35}$$

Recall that the solution is obtained by assuming that the homogeneous solution has the form

$$y_h(n) = \lambda^n$$

Substitution of this solution into (7.4.35) yields the characteristic polynomial

$$\lambda^2 - \lambda - 1 = 0$$

But this is exactly the same characteristic polynomial obtained from the determinant of $(\mathbf{F} - \lambda\mathbf{I})$.

Since the state-variable realization of the system is not unique, the matrix F is also not unique. However, the eigenvalues of the system are unique, that is, they are invariant to any nonsingular linear transformation of F. Consequently, the characteristic polynomial of F can be determined either from evaluating the determinant of $(\mathbf{F} - \lambda\mathbf{I})$ or from the difference equation characterizing the system.

In conclusion, the state-space description provides an alternative characterization of the system that is equivalent to the input-output description. One advantage of the state-variable formulation is that it provides us with the additional information concerning the internal (state) variables of the system, information that is not easily obtained from the input-output description. Furthermore, the state-variable formulation of a linear time-invariant system allows us to represent the system by a set of (usually coupled) first-order difference equations. The decoupling of the equations can be achieved by means of a linear transformation that can be obtained by solving for the eigenvalues and eigenvectors of the system. The decoupled equations are then relatively simple to solve. More important, however, the state-space formulation provides a powerful, yet straightforward method for dealing with systems that have multiple inputs and multiple outputs **(MIMO).** Although we have not considered such systems in our study, it is in the treatment of **MIMO** systems where the true power and the beauty of the space-space formulation can be fully appreciated.

7.4.4 State-Space Analysis in the z-Domain

The state-space analysis in the previous sections has been performed in the time domain. However, as we have observed previously, the analysis of linear time-invariant discrete-time systems can also be carried out in the z-transform

domain, often with greater ease. In this section we treat the state-space representation of linear time-invariant discrete-time systems in the z-transform domain.

Let us consider the state-space equation

$$\mathbf{v}(n+1) = \mathbf{F}\mathbf{v}(n) + \mathbf{q}x(n) \tag{7.4.36}$$

If we define the vector $\mathbf{V}(z)$ as

$$\mathbf{V}(z) = \begin{bmatrix} V_1(z) \\ V_2(z) \\ \vdots \\ V_N(z) \end{bmatrix} \tag{7.4.37}$$

then (7.4.36) can be expressed in matrix form as

$$z\mathbf{V}(z) = \mathbf{F}\mathbf{V}(z) + \mathbf{q}X(z) \tag{7.4.38}$$

The two terms involving $\mathbf{V}(z)$ can be collected together and the resulting equation can be used to solve for $\mathbf{V}(z)$. Thus

$$\begin{aligned} (z\mathbf{I} - \mathbf{F})\mathbf{V}(z) &= \mathbf{q}X(z) \\ \mathbf{V}(z) &= (z\mathbf{I} - \mathbf{F})^{-1}\mathbf{q}X(z) \end{aligned} \tag{7.4.39}$$

The inverse z-transform of (7.4.39) yields the solution for the state equations.

Next, we turn our attention to the output equation, which is given as

$$y(n) = \mathbf{g}'\mathbf{v}(n) + dx(n) \tag{7.4.40}$$

The z-transform of (7.4.40) is

$$Y(z) = \mathbf{g}'\mathbf{V}(z) + dX(z) \tag{7.4.41}$$

By using the solution in (7.4.39) we can eliminate the state vector $\mathbf{V}(z)$ in (7.4.41). Thus we obtain

$$Y(z) = [\mathbf{g}'(z\mathbf{I} - \mathbf{F})^{-1}\mathbf{q} + d]X(z) \tag{7.4.42}$$

which is the z-transform of the zero-state response of the system. The system function is easily obtained from (7.4.42) as

$$H(z) = \frac{Y(z)}{X(z)} = \mathbf{g}'(z\mathbf{I} - \mathbf{F})^{-1}\mathbf{q} + d \tag{7.4.43}$$

The state equation given by (7.4.39), the output equation given by (7.4.42) and the system function given by (7.4.43) all have in common the factor $(z\mathbf{I} - \mathbf{F})^{-1}$. This is a fundamental quantity that is related to the z-transform of the state transition matrix of the system. The relationship is easily established by computing the

z-transform of the impulse response $h(n)$, which is given by (7.4.28). Thus we have

$$\begin{aligned} H(z) &= \sum_{n=0}^{\infty} h(n)z^{-n} \\ &= \sum_{n=0}^{\infty} [\mathbf{g}'\mathbf{F}^{n-1}\mathbf{q}u(n-1) + d\delta(n)]z^{-n} \\ &= \mathbf{g}' \left(\sum_{n=1}^{\infty} \mathbf{F}^{n-1}z^{-n} \right) \mathbf{q} + d \end{aligned} \tag{7.4.44}$$

The term in parentheses in (7.4.44) can be written as

$$\begin{aligned} \sum_{n=1}^{\infty} \mathbf{F}^{n-1}z^{-n} &= z^{-1}(\mathbf{I} + \mathbf{F}z^{-1} + \mathbf{F}^2z^{-2} + \cdots) \\ &= z^{-1}(\mathbf{I} - \mathbf{F}z^{-1})^{-1} = (z\mathbf{I} - \mathbf{F})^{-1} \end{aligned} \tag{7.4.45}$$

If we substitute the result in (7.4.45) into (7.4.44), we obtain the expression for $H(z)$ as given in (7.4.43).

Since the state transition matrix is given by

$$(n) = \mathbf{F} \tag{7.4.46}$$

the z-transform of (n) is

$$\begin{aligned} \sum_{n=0}^{\infty} \mathbf{F}^n z^{-n} &= \mathbf{I} + \mathbf{F}z^{-1} + \mathbf{F}^2z^{-2} + \mathbf{F}^3z^{-3} + \cdots \\ &= (\mathbf{I} - \mathbf{F}z^{-1})^{-1} = z(z\mathbf{I} - \mathbf{F})^{-1} \end{aligned} \tag{7.4.47}$$

The relation in (7.4.47) provides a simple method for determining the state transition matrix by means of z-transforms. We recall that

$$(z\mathbf{I} - \mathbf{F})^{-1} = \frac{\text{adj}(z\mathbf{I} - \mathbf{F})}{\det(z\mathbf{I} - \mathbf{F})} \tag{7.4.48}$$

where adj(A) denotes the *adjoint matrix* of A and det (A) denotes the determinant of the matrix A. Substitution of (7.4.48) into (7.4.43) yields the result

$$H(z) = \mathbf{g}' \frac{\text{adj}(z\mathbf{I} - \mathbf{F})}{\det(z\mathbf{I} - \mathbf{F})} \mathbf{q} + d \tag{7.4.49}$$

Consequently, the denominator $D(z)$ of the system function $H(z)$, which contains the poles of the system is simply

$$D(z) = \det(z\mathbf{I} - \mathbf{F}) \tag{7.4.50}$$

But the det(zI − F) is just the characteristic polynomial of F. Its roots, which are the poles of system, are the eigenvalues of the matrix F.

Example 7.4.3

Determine the system function $H(z)$, the impulse response $h(n)$, and the state transition matrix $\mathbf{\Phi}(n)$ of the system that generates the Fibonacci sequence. This system is described by the state-space equation

$$\mathbf{v}(n+1) = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \mathbf{v}(n) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} x(n)$$

$$y(n) = \begin{bmatrix} 1 & 1 \end{bmatrix} \mathbf{v}(n) + x(n)$$

Solution First, we determine $H(z)$ and $h(n)$ by computing $(z\mathbf{I} - \mathbf{F})^{-1}$. We have

$$(z\mathbf{I} - \mathbf{F})^{-1} = \begin{bmatrix} z & -1 \\ -1 & z-1 \end{bmatrix}^{-1} = \frac{1}{z^2 - z - 1} \begin{bmatrix} z-1 & 1 \\ 1 & z \end{bmatrix}$$

Hence

$$H(z) = \frac{1}{z^2 - z - 1} \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} z-1 & 1 \\ 1 & z \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 1$$

$$= \frac{z^2}{z^2 - z - 1} = \frac{1}{1 - z^{-1} - z^{-2}}$$

By inverting $H(z)$, we obtain $h(n)$ in the form

$$h(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{n+1} - \left(\frac{1-\sqrt{5}}{2} \right)^{n+1} \right] u(n)$$

We note that the poles of $H(z)$ are $p_1 = (1 + \sqrt{5})/2$ and $p_2 = (1 - \sqrt{5})/2$. Since $|p_1| > 1$, the system that generates the Fibonacci sequence is unstable.

The state transition matrix $\mathbf{\Phi}(n)$ has the z-transform

$$z(z\mathbf{I} - \mathbf{F})^{-1} = \frac{1}{z^2 - z - 1} \begin{bmatrix} z^2 - z & z \\ z & z^2 \end{bmatrix}$$

The four elements of $\mathbf{\Phi}(n)$ are obtained by computing the inverse transform of the four elements of $z(z\mathbf{I} - \mathbf{F})^{-1}$. Thus we obtain

$$\mathbf{\Phi}(n) = \begin{bmatrix} \phi_{11}(n) & \phi_{12}(n) \\ \phi_{21}(n) & \phi_{22}(n) \end{bmatrix}$$

where

$$\phi_{11}(n) = \left[\frac{1+\sqrt{5}}{2\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n - \frac{1-\sqrt{5}}{2\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n \right] u(n)$$

$$\phi_{12}(n) = \phi_{21}(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right] u(n)$$

$$\phi_{22}(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{n+1} - \left(\frac{1-\sqrt{5}}{2} \right)^{n+1} \right] u(n)$$

We note that the impulse response $h(n)$ can also be computed from (7.4.28) by using the state transition matrix.

This analysis method appiies specifically to the computation of the zero-state response of the system. This is the consequence of the fact that we have used the two-sided z-transform.

If we wish to determine the total response of the system, beginning at a nonzero state, say $\mathbf{v}(n_0)$, we must use the one-sided z-transform. Thus, for a given initial state $\mathbf{v}(n_0)$ and a given input $x(n)$ for $n \geq n_0$, we can determine the state vector $\mathbf{v}(n)$ for $n \geq n_0$ and the output $y(n)$ for $n \geq n_0$, by means of the one-sided z-transform.

In this development we assume that $n_0 = 0$, without loss of generality. Then, given $x(n)$ for $n \geq 0$, and a causal system, described by the state equations in (7.4.36), the one-sided z-transform of the state equations is

$$z\mathbf{V}^+(z) - z\mathbf{v}(0) = \mathbf{F}\mathbf{V}^+(z) + \mathbf{q}X(z)$$

or, equivalently,

$$\mathbf{V}^+(z) = z(z\mathbf{I} - \mathbf{F})^{-1}\mathbf{v}(0) + (z\mathbf{I} - \mathbf{F})^{-1}\mathbf{q}X(z) \tag{7.4.51}$$

Note that $X^+(z) = X(z)$, since $x(n)$ is assumed to be causal.

Similarly, the z-transform of the output equation given by (7.4.40) is

$$Y^+(z) = \mathbf{g}'\mathbf{V}^+(z) + dX(z) \tag{7.4.52}$$

If we substitute for $\mathbf{V}^+(z)$ from (7.4.51) into (7.4.52), we obtain the result

$$Y^+(z) = z\mathbf{g}'(z\mathbf{I} - \mathbf{F})^{-1}\mathbf{v}(0) + [\mathbf{g}'(z\mathbf{I} - \mathbf{F})^{-1}\mathbf{q} + d]X(z) \tag{7.4.53}$$

Of the terms on the right-hand side of (7.4.53), the first represents the zero-input response of the system due to the initial conditions, while the second represents the zero-state response of the system that we obtained previously. Consequently, (7.4.53) constitutes the total response of the system, which can be expressed in the time domain by inverting (7.4.53). The result of this inversion yields the form for $y(n)$ given previously by (7.4.16).

7.4.5 Additional State-Space Structures

In Section 7.4.2 we described how state-space equations can be obtained from a given structure and, conversely, how to obtain a realization of the system given the state equations. In this section we revisit the parallel-form and cascade-form realizations described previously and consider these structures in the context of a state-space formulation.

The parallel-form state-space structure is obtained by expanding the system function $H(z)$ into a partial-fraction expansion, developing the state-space formulation for each term in the expansion and the corresponding structure, and finally, connecting all the structures in parallel. We illustrate the procedure under the assumption that the poles are distinct and $N = M$.

The system function $H(z)$ can be expressed as

$$H(z) = C + \sum_{k=1}^{N} \frac{B_k}{z - p_k} \tag{7.4.54}$$

Note that this is a different expansion from that given in (7.3.20). The output of the system is

$$Y(z) = H(z)X(z) = CX(z) + \sum_{k=1}^{N} B_k Y_k(z) \tag{7.4.55}$$

where, by definition.

$$Y_k(z) = \frac{X(z)}{z - p_k} \qquad k = 1, 2, \ldots, N \tag{7.4.56}$$

In the time domain, the equations in (7.4.56) become

$$y_k(n+1) = p_k y_k(n) + x(n) \qquad k = 1, 2, \ldots, N \tag{7.4.57}$$

We define the state variables as

$$v_k(n) = y_k(n) \qquad k = 1, 2, \ldots, N \tag{7.4.58}$$

Then the difference equations in (7.4.57) become

$$v_k(n+1) = p_k v_k(n) + x(n) \qquad k = 1, 2, \ldots, N \tag{7.4.59}$$

The state equations in (7.4.59) can be expressed in matrix form as

$$\mathbf{v}(n+1) = \begin{bmatrix} p_1 & 0 & \cdots & 0 \\ 0 & p_2 & \cdots & 0 \\ & & \ddots & \\ 0 & & & p_N \end{bmatrix} \mathbf{v}(n) + \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} x(n) \tag{7.4.60}$$

and the output equation is

$$y(n) = [\, B_1 \quad B_2 \quad \cdots \quad B_N \,]\, \mathbf{v}(n) + Cx(n) \tag{7.4.61}$$

This parallel-form realization is called the normal *form* representation, because the matrix F is diagonal, and hence the state variables are uncoupled. An alternative structure is obtained by pairing complex-conjugate poles and any two real-valued poles to form second-order sections, which can be realized by using either type 1 or type 2 state-space structures.

The cascade-form state-space structure can be obtained by factoring H(z) into a product of first-order and second-order sections, as described in Section 7.2.2, and then implementing each section by using either type 1 or type 2 state-space structures.

Let us consider the state-space representation of a single second-order section involving a pair of complex-conjugate poles. The system function is

$$\begin{aligned} H(z) &= \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \\ &= \frac{b_0 z^2 + b_1 z + b_2}{z^2 + a_1 z + a_2} \\ &= b_0 + \frac{A}{z - p} + \frac{A^*}{z - p^*} \end{aligned} \tag{7.4.62}$$

The output of this system can be expressed as

$$Y(z) = b_0 X(z) + \frac{AX(z)}{z - p} + \frac{A^* X(z)}{z - p^*} \tag{7.4.63}$$

We define the quantity

$$S(z) = \frac{AX(z)}{z - p} \tag{7.4.64}$$

This relationship can be expressed in the time domain as

$$s(n+1) = ps(n) + Ax(n) \tag{7.4.65}$$

Since $s(n)$, p, and A are complex valued, we define $s(n)$ as

$$\begin{aligned} s(n) &= v_1(n) + jv_2(n) \\ p &= \alpha_1 + j\alpha_2 \\ A &= q_1 + jq_2 \end{aligned} \tag{7.4.66}$$

Upon substitution of these relations into (7.4.65) and separating its real and imaginary parts, we obtain

$$\begin{aligned} v_1(n+1) &= \alpha_1 v_1(n) - \alpha_2 v_2(n) + q_1 x(n) \\ v_2(n+1) &= \alpha_2 v_1(n) + \alpha_1 v_2(n) + q_2 x(n) \end{aligned} \tag{7.4.67}$$

We choose $v_1(n)$ and $v_2(n)$ as the state variables and thus obtain the coupled pair of state equations which can be expressed in matrix form as

$$\mathbf{v}(n+1) = \begin{bmatrix} \alpha_1 & -\alpha_2 \\ \alpha_2 & \alpha_1 \end{bmatrix} \mathbf{v}(n) + \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} x(n) \tag{7.4.68}$$

The output equation can be expressed as

$$y(n) = b_0 x(n) + s(n) + s^*(n) \tag{7.4.69}$$

Upon substitution for $s(n)$ in (7.4.69), we obtain the desired result for the output in the form

$$y(n) = [2 \quad 0]\mathbf{v}(n) + b_0 x(n) \tag{7.4.70}$$

A realization for the second-order section is shown in Fig. 7.31. It is simply called the coupled-form state-space realization. This structure, which is used as the building block in the implementation of cascade-form realizations for higher-order IIR systems, exhibits low sensitivity to finite-word-length effects.

7.5 REPRESENTATION OF NUMBERS

Up to this point we have considered the implementation of discrete-time systems without being concerned about the finite-word-length effects that are inherent in any digital realization, whether it be in hardware or in software. In fact, we have analyzed systems that are modeled as linear when, in fact, digital realizations of such systems are inherently nonlinear.

In this and the following two sections, we consider the various forms of quantization effects that arise in digital signal processing, Although we describe

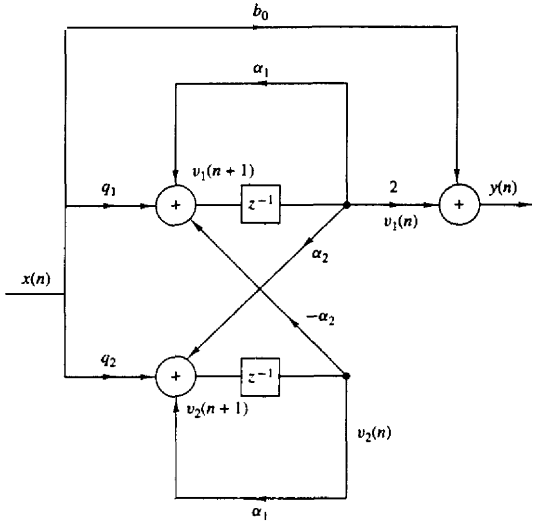


Figure 7.31 Coupled-form state-space realization of a two-pole, two-zero IIR system.

floating-point arithmetic operations briefly, our major concern is with fixed-point realizations of digital filters.

In this section we consider the representation of numbers for digital computations. The main characteristic of digital arithmetic is the limited (usually fixed) number of digits used to represent numbers. This constraint leads to finite numerical precision in computations, which leads to round-off errors and nonlinear effects in the performance of digital filters. We now provide a brief introduction to digital arithmetic.

7.5.1 Fixed-Point Representation of Numbers

The representation of numbers in a fixed-point format is a generalization of the familiar decimal representation of a number as a string of digits with a decimal point. In this notation, the digits to the left of the decimal point represent the integer part of the number, and the digits to the right of the decimal point represent the fractional part of the number. Thus a real number X can be represented as

$$\begin{aligned} X &= (b_{-A}, \ldots, b_{-1}, b_0, b_1, \ldots, b_B)_r \\ &= \sum_{i=-A}^{B} b_i r^{-i} \qquad 0 \le b_i \le (r-1) \end{aligned} \tag{7.5.1}$$

where b_i represents the digit, r is the radix or base, A is the number of integer

digits, and B is the number of fractional digits. As an example, the decimal number $(123.45)_{10}$ and the binary number $(101.01)_2$ represent the following sums:

$$(123.45)_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

$$(101.01)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$

Let us focus our attention on the binary representation since it is the most important for digital signal processing. In this case $r = 2$ and the digits $\{b_i\}$ are called binary digits or bits and take the values $\{0,1\}$. The binary digit b_{-A} is called the most significant bit (MSB) of the number, and the binary digit b_B is called the least significant bit (LSB). The "binary point" between the digits b_0 and b_1 does not exist physically in the computer. Simply, the logic circuits of the computer are designed so that the computations result in numbers that correspond to the assumed location of this point.

By using an n-bit integer format (A = n − 1, B = 0), we an represent unsigned integers with magnitude in the range 0 to $2^n - 1$. Usually, we use the fraction format ($A = 0$, $B = n - 1$), with a binary point between b_0 and b_1, that permits numbers in the range from 0 to $1 - 2^{-n}$. *Note* that any integer or mixed number can be represented in a fraction format by factoring out the term r^A in (7.5.1). In the sequel we focus our attention on the binary fraction format because mixed numbers are difficult to multiply and the number of bits representing an integer cannot be reduced by truncation or rounding.

There are three ways to represent negative numbers. This leads to three formats for the representation of signed binary fractions. The format for positive fractions is the same in all three representations, namely,

$$X = 0.b_1b_2 \cdots b_B = \sum_{i=1}^{B} b_i \cdot 2^{-i}, \quad X \geq 0 \tag{7.5.2}$$

Note that the MSB b_0 is set to zero to represent the positive sign. Consider now the negative fraction

$$X = -0.b_1b_2 \cdots b_B = -\sum_{i=1}^{B} b_i \cdot 2^{-i} \tag{7.5.3}$$

This number can be represented using one of the following three formats.

Sign-magnitude format. In this format, the MSB is set to 1 to represent the negative sign,

$$X_{SM} = 1.b_1b_2 \cdots b_B \qquad \text{for } X \leq 0 \tag{7.5.4}$$

One's-complement format. In this format the negative numbers are represented as

$$X_{1C} = 1.\bar{b}_1\bar{b}_2 \cdots \bar{b}_B \qquad X \leq 0 \tag{7.5.5}$$

where $\bar{b}_i = 1 - b_i$ is the one's complement of b_i. Thus if X is a positive number, the corresponding negative number is determined by complementing (changing 1's

to 0's and 0's to 1's) all the bits. An alternative definition for X_{1C} can be obtained by noting that

$$X_{1C} = 1 \times 2^0 + \sum_{i=1}^{B} (1 - b_i) \cdot 2^{-i} = 2 - 2^{-B}|X| \tag{7.5.6}$$

Two's-complement format. In this format a negative number is represented by forming the two's complement of the corresponding positive number. In other words, the negative number is obtained by subtracting the positive number from 2.0. More simply, the two's complement is formed by complementing the positive number and adding one LSB. Thus

$$X_{2C} = 1.\bar{b}_1\bar{b}_2 \cdots \bar{b}_B + 00 \cdots 01 \qquad X < 0 \tag{7.5.7}$$

where $+$ represents modulo-2 addition that ignores any carry generated from the sign bit. For example, the number $-\frac{3}{8}$ is simply obtained by complementing 0011 ($\frac{3}{8}$) to obtain 1100 and then adding 0001. This yields 1101. which represents $-\frac{3}{8}$ in two's complement.

From (7.5.6) and (7.5.7) is can easily be seen that

$$X_{2C} = X_{1C} + 2^{-B} = 2 - |X| \tag{7.5.8}$$

To demonstrate that (7.5.7) truly represents a negative number, we use the identity

$$1 = \sum_{i=1}^{B} 2^{-i} + 2^{-B} \tag{7.5.9}$$

The negative number X in (7.5.3) can be expressed as

$$\begin{aligned} X_{2C} &= -\sum_{i=1}^{B} b_i \cdot 2^{-i} + 1 - 1 \\ &= -1 + \sum_{i=1}^{B} (1 - b_i) 2^{-i} + 2^{-B} \\ &= -1 + \sum_{i=1}^{B} \bar{b}_i \cdot 2^{-1} + 2^{-B} \end{aligned}$$

which is exactly the two's-complement representation of (7.5.7).

In summary, the value of a binary string $b_0 b_1 \ldots b_B$ depends on the format used. For positive numbers, $b_0 = 0$, and the number is given by (7.5.2). For negative numbers, we use these corresponding formulas for the three formats.

Example 7.5.1

Express the fraction $\frac{7}{8}$ and $-\frac{7}{8}$ in sign-magnitude, two's-complement, and one's-complement format.

Solution $X = \frac{7}{8}$ **is represented as** $2^{-1} + 2^{-2} + 2^{-3}$, **so that** X = 0.111. **In sign-magnitude format,** $X = -\frac{7}{8}$ **is represented as** 1.111. **In one's complement. we have**

$$X_{1C} = 1.000$$

In two's complement. the result is

$$X_{2C} = 1.000 + 0.001 = 1.001$$

The basic arithmetic operations of addition and multiplication depend on the format used. For one's-complement and two's-complement formats, addition is carried out by adding the numbers bit by bit. The formats differ only in the way in which a carry bit affects the **MSB.** For example, $\frac{4}{8} - \frac{3}{8} = \frac{1}{8}$. In two's complement, we have

$$0100\$1101 = 0001$$

where $\oplus$ indicates modulo-2 addition. Note that the carry bit. if present in the **MSB.** is dropped. On the other hand. in one's- complement arithmetic, the carry in the MSB, if present, is carried around to the LSB. Thus the computation $\frac{4}{8} - \frac{3}{8} = \frac{1}{8}$ becomes

$$0100 \oplus 1100 = 0000 \oplus 0001 = 0001$$

Addition in the sign-magnitude format is more complex and can involve sign checks, complementing, and the generation of a carry. On the other hand, direct multiplication of two sign- magnitude numbers is relatively straightforward, whereas a special algorithm is usually employed for one's complement and two's complement multiplication.

Most fixed-point digital signal processors use two's-complement arithmetic. Hence, the range for $(B+1)$–bit numbers is from -1 to $1-2^{-B}$. These numbers can be viewed in a wheel format as shown in Fig. 7.32 for $B = 2$. Two's-complement arithmetic is basically arithmetic **modulo-**2^{B+1} [i.e., any number that falls outside

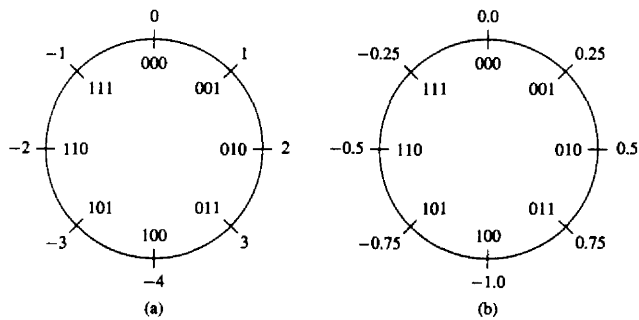


Figure 7.32 Counting wheel for 3-bit two's-complement numbers (a) integers and (b) functions.

the range (overflow or underflow) is reduced to this range by subtracting an appropriate multiple of 2^{B+1}]. This type of arithmetic can be viewed as counting using the wheei of Fig. 7.32. A very important property of two's- complement addition is that if the final sum of a string of numbers $X_1, X_2, \ldots, X_N$ is within the range, it will be computed correctly, even if individual partial sums result in overflows. This and other characteristics of two's-complement arithmetic are considered in Problem 7.44.

In general, the multiplication of two fixed-point numbers each of b bits in length results in a product of 2b bits in length. In fixed-point arithmetic, the product is either truncated or rounded back to b bits. As a result we have a truncation or round-off error in the b least significant bits. The characterization of such errors is treated below.

7.5.2 Binary Floating-Point Representation of Numbers

A fixed-point representation of numbers allows us to cover a range of numbers, say, $x_{\max} - x_{\min}$ with a resolution

$$\Delta = \frac{x_{\max} - x_{\min}}{m - 1}$$

where $m = 2^b$ is the number of levels and b is the number of bits. A basic characteristic of the fixed-point representation is that the resolution is fixed. Furthermore, A increases in direct proportion to an increase in the dynamic range.

A floating-point representation can be employed as a means for covering a larger dynamic range. The binary floating-point representation commonly used in practice, consists of a mantissa M, which is the fractional part of the number and falls in the range $\frac{1}{2} \leq M < 1$, multiplied by the exponential factor 2^E, where the exponent E is either a positive or negative integer. Hence a number X is represented as

$$X = M \cdot 2^E$$

The mantissa requires a sign bit for representing positive and negative numbers, and the exponent requires an additional sign bit. Since the mantissa is a signed fraction, we can use any of the four fixed-point representations just described.

For example, the number $X_1 = 5$ is represented by the following mantissa and exponent:

$$M_1 = 0.101000$$

$$E_1 = 011$$

while the number $X_2 = \frac{3}{8}$ is represented by the following mantissa and exponent

$$M_2 = 0.110000$$

$$E_2 = 101$$

where the leftmost bit in the exponent represents the sign bit.

If the two numbers are to be multiplied, the mantissas are multiplied and the exponents are added. Thus the product of these two numbers is

$$\begin{aligned} X_1X_2 &= M_1M_2 \cdot 2^{E_1+E_2} \\ &= (0.011110) \cdot 2^{010} \\ &= (0.111100) \cdot 2^{001} \end{aligned}$$

On the other hand, the addition of the two floating-point numbers requires that the exponents be equal. This can be accomplished by shifting the mantissa of the smaller number to the right and compensating by increasing the corresponding exponent. Thus the number X_2 can be expressed as

$$M_2 = 0.000011$$

$$E_2 = 011$$

With $E_2 = E_1$, we can add the two numbers X_1 and X_2. The result is

$$X_1 + X_2 = (0.101011) \cdot 2^{011}$$

It should be observed that the shifting operation required to equalize the exponent of X_2 with that for X_1 results in loss of precision, in general. In this example the six-bit mantissa was sufficiently long to accommodate a shift of four bits to the right for M_2 without dropping any of the ones. However, a shift of five bits would have caused the loss of a single bit and a shift of six bits to the right would have resulted in a mantissa of $M_2 = 0.000000$, unless we round upward after shifting so that $M_2 = 0.000001$.

Overflow occurs in the multiplication of two floating-point numbers when the sum of the exponents exceeds the dynamic range of the fixed-point representation of the exponent.

In comparing a fixed-point representation with a floating-point representation, each with the same number of total bits, it is apparent that the floating-point representation allows us to cover a larger dynamic range by varying the resolution across the range. The resolution decreases with an increase in the size of successive numbers. In other words, the distance between two successive floating-point numbers increases as the numbers increase in size. It is this variable resolution that results in a larger dynamic range. Alternatively, if we wish to cover the same dynamic range with both fixed-point and floating-point representations, the floating-point representation provides finer resolution for small numbers but coarser resolution for the larger numbers. In contrast, the fixed-point representation provides a uniform resolution throughout the range of numbers.

For example, if we have a computer with a word size of 32 bits, it is possible to represent 2^{32} numbers. If we wish to represent the positive integers beginning with zero, the largest possible integer that can be accommodated is

$$2^{32} - 1 = 4{,}294{,}967{,}295$$

The distance between successive numbers (the resolution) is 1. Alternatively, we can designate the leftmost bit as the sign bit and use the remaining 31 bits for the magnitude. In such a case a fixed-point representation allows us to cover the range

$$-(2^{31} - 1) = -2{,}147{,}483{,}647 \quad \text{to} \quad (2^{31} - 1) = 2{,}147{,}483{,}647$$

again with a resolution of 1.

On the other hand, suppose that we increase the resolution by allocating 10 bits for a fractional part, 21 bits for the integer part, and 1 bit for the sign. Then this representation allows us to cover the dynamic range

$$-(2^{31} - 1) \cdot 2^{-10} = -(2^{21} - 2^{-10}) \quad \text{to} \quad (2^{31} - 1) \cdot 2^{-10} = 2^{21} - 2\text{-}10$$

or, equivalently,

$$-2{,}097{,}151.999 \quad \text{to} \quad 2{,}097{,}151.999$$

In this case, the resolution is 2^{-10}. Thus, the dynamic range has been decreased by a factor of approximately 1000 (actually 2^{10}), while the resolution has been increased by the same factor.

For comparison, suppose that the 32-bit word is used to represent floating-point numbers. In particular, let the mantissa be represented by 23 bits plus a sign bit and let the exponent be represented by 7 bits plus a sign bit. Now, the smallest number in magnitude will have the representation,

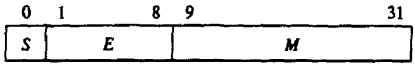
sign	23 bits	sign	7 bits	
0	$100\cdots0$	1	1111111	$= \frac{1}{2} \times 2^{-127} \approx 0.3 \times 10^{-38}$

At the other extreme, the largest number that can be represented with this floating-point representation is

sign	23 bits	sign	7 bits	
0	$111\cdots1$	0	1111111	$= (1 - 2^{-23}) \times 2^{127} \approx 1.7 \times 10^{38}$

Thus, we have achieved a dynamic range of approximately 10^{76}, but with varying resolution. In particular, we have fine resolution for small numbers and coarse resolution for larger numbers.

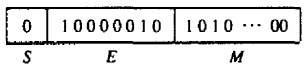
The representation of zero poses some special problems. In general, only the mantissa has to be zero, but not the exponent. The choice of M and E, the representation of zero, the handling of overflows, and other related issues have resulted in various floating-point representations on different digital computers. In an effort to define a common floating-point format, the Institute of Electrical and Electronic Engineers (IEEE) introduced the IEEE 754 standard, which is widely used in practice. For a 32-bit machine, tbe IEEE 754 standard single-precision, floating-point number is represented as $X = (-1)^s \cdot 2^{E-127}(M)$, where



This number has the following interpretations:

If $E = 255$ and $M \neq 0$, then X is not a number
If $E = 255$ and $M = 0$, then $X = (-1)^S \cdot \infty$
If $0 < E < 255$, then $X = (-1)^S \cdot 2^{E-127}(1.M)$
If $E = 0$ and $M \neq 0$, then $X = (-1)^S \cdot 2^{-126}(0.M)$
If $E = 0$ and $M = 0$, then $X = (-1)^S \cdot 0$

where $0.M$ is a fraction and $1.M$ is a mixed number with one integer bit and 23 fractional bits. For example, the number



has the value $X = -1^0 \times 2^{130-127} \times 1.1010...0 = 2^3 \times \frac{13}{8} = 13$. The magnitude range of the 32-bit IEEE 754 floating-point numbers is from $2^{-126} \times 2^{-23}$ to $(2-2^{-23}) \times 2^{127}$ (i.e., from 1.18×10^{-38} to 3.40×10^{38}). Computations with numbers outside this range result in either underflow or overflow.

7.5.3 Errors Resulting from Rounding and Truncation

In performing computations such as multiplications with either fixed-point or floating-point arithmetic, we are usually faced with the problem of quantizing a number via truncation or rounding, from a given level of precision to a level of lower precision. The effect of rounding and truncation is to introduce an error whose value depends on the number of bits in the original number relative to the number of bits after quantization. The characteristics of the errors introduced through either truncation or rounding depend on the particular form of number representation.

To be specific, let us consider a fixed-point representation in which a number x is quantized from b_u bits to b bits. Thus the number

$$x = \overbrace{0.1011\cdots01}^{b_u}$$

consisting of b_u bits prior to quantization is represented as

$$x = \overbrace{0.101\cdots1}^{b}$$

after quantization, where $b < b_u$. For example, if x represents the sample of an analog signal, then b_u may be taken as infinite. In any case if the quantizer truncates the value of x, the truncation error is defined as

$$E_t = Q_t(x) - x \tag{7.5.10}$$

First, we consider the range of values of the error for sign-magnitude and two's-complement representation. In both of these representations, the positive numbers have identical representations. For positive numbers, truncation results in a number that is smaller than the unquantized number. Consequently, the truncation error resulting from a reduction of the number of significant bits from b_u to b is

$$-(2^{-b} - 2^{-b_u}) \le E_t \le 0 \tag{7.5.11}$$

where the largest error arises from discarding $b_u - b$ bits, all of which are ones.

In the case of negative fixed-point numbers based on the sign-magnitude representation, the truncation error is positive, since truncation basically reduces the magnitude of the numbers. Consequently, for negative numbers, we have

$$0 \le E_t \le (2^{-b} - 2^{-b_u}) \tag{7.5.12}$$

In the two's-complement representation, the negative of a number is obtained by subtracting the corresponding positive number from 2. As a consequence, the effect of truncation on a negative number is to increase the magnitude of the negative number. Consequently, $x > Q_t(x)$ and hence

$$-(2^{-b} - 2^{-b_u}) \le E_t \le 0 \tag{7.5.13}$$

Hence we conclude that ***the truncation error for the sign-magnitude representation is symmetric about zero and falls in the range***

$$-(2^{-b} - 2^{-b_u}) \le E_t \le (2^{-b} - 2^{-b_u}) \tag{7.5.14}$$

On the other hand, ***for two's-complement representation, the truncation error is always negative and falls in the range***

$$-(2^{-b} - 2^{-b_u}) \le E_t \le 0 \tag{7.5.15}$$

Next, let us consider the quantization errors due to rounding of a number. A number x, represented by b_u bits before quantization and b bits after quantization, incurs a quantization error

$$E_r = Q_r(x) - x \tag{7.5.16}$$

Basically, rounding involves only the magnitude of the number and, consequently, the round-off error is independent of the type of fixed-point representation. The maximum error that can be introduced through rounding is $(2^{-b} - 2^{-b_u})/2$ and this can be either positive or negative, depending on the value of x. Therefore, ***the round-off error is symmetric about zero and falls in the range***

$$-\tfrac{1}{2}(2^{-b} - 2^{-b_u}) \le E_r \le \tfrac{1}{2}(2^{-b} - 2^{-b_u}) \tag{7.5.17}$$

These relationships are summarized in Fig. 7.33 when x is a continuous signal amplitude (b, $= \infty$).

In a **floating-point** representation, the mantissa is either rounded or truncated. Due to the **nonuniform** resolution, the corresponding error in a floating-point representation is proportional to the number being quantized. An appropriate

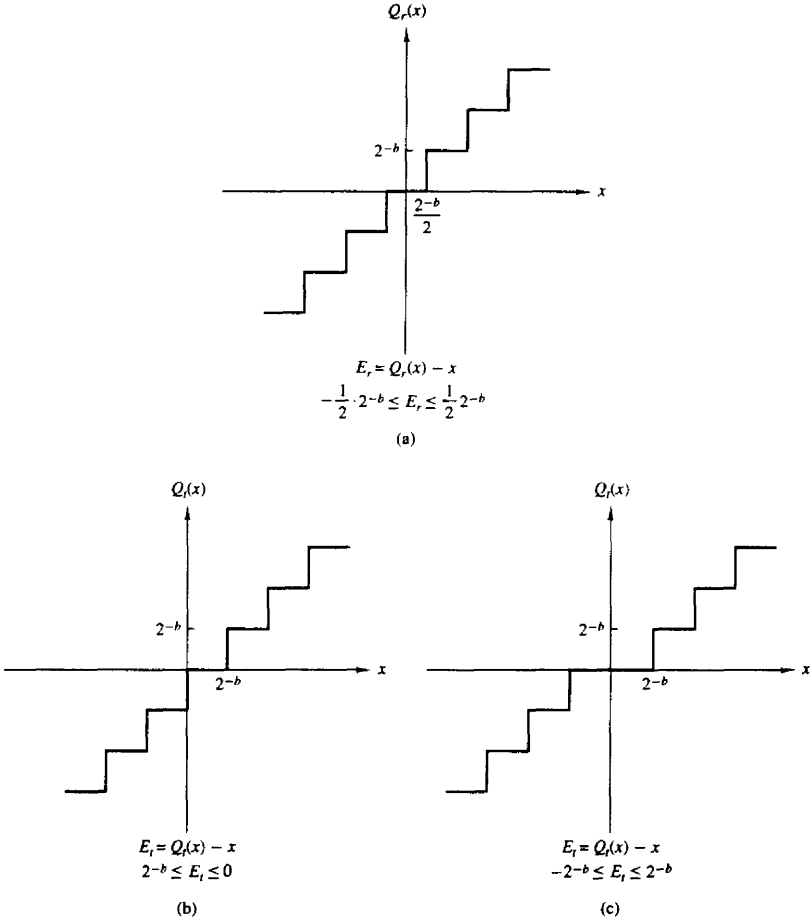


Figure 7.33 Quantization errors in rounding and truncation: (a) rounding; (b) truncation in two's complement; (c) truncation in sign-magnitude.

representation for the quantized value is

$$Q(x) = x + ex \tag{7.5.18}$$

where e is called the relative error. Now

$$Q(x) - x = ex \tag{7.5.19}$$

In the case of truncation based on two's-complement representation of the mantissa, we have

$$-2^E 2^{-b} < e_t x < 0 \tag{7.5.20}$$

for positive numbers. Since $2^{E-1} \leq x < 2^E$, it follows that

$$-2^{-b+1} < e_t \leq 0 \qquad x > 0 \tag{7.5.21}$$

On the other hand, for a negative number in two's-complement representation, the error is

$$0 \leq e_t x < 2^E 2^{-b}$$

and hence

$$0 \leq e_t < 2^{-b+1} \qquad x < 0 \tag{7.5.22}$$

In the case where the mantissa is rounded, the resulting error is symmetric relative to zero and has a maximum value of $\pm 2^{-b}/2$. Consequently, the round-off error becomes

$$-2^E \cdot 2^{-b}/2 < e_r x \leq 2^E \cdot 2^{-b}/2 \tag{7.5.23}$$

Again, since x falls in the range $2^{E-1} \leq x < 2^E$, we divide through by 2^{E-1} so that

$$-2^{-b} < e_r \leq 2^{-b} \tag{7.5.24}$$

In arithmetic computations involving quantization via truncation and rounding, it is convenient to adopt a statistical approach to the characterization of such errors. The quantizer can be modeled as introducing an additive noise to the unquantized value x. Thus we can write

$$Q(x) = x + \epsilon$$

where $\epsilon = E_r$ for rounding and $\epsilon = \mathrm{E}$, for truncation. This model is illustrated in Fig. 7.34.

Since x can be any number that falls within any of the levels of the quantizer, the quantization error is usually modeled as a random variable that falls

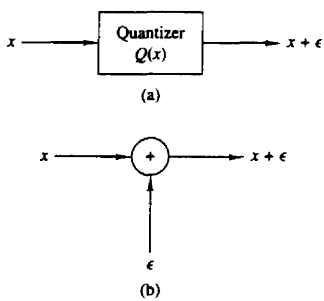


Figure 7.34 Additive noise model for the nonlinear quantization process: (a) actual system; (b) model for quantization.

within the limits specified. This random variable is assumed to be uniformly distributed within the ranges specified for the fixed-point representations. Furthermore, in practice, $b_u >> b$, so that we can neglect the factor of 2^{-b_u} in the formulas given below. Under these conditions, the probability density functions for the round-off and truncation errors in the two fixed-point representations are illustrated in Fig. 7.35. We note that in the case of truncation of the two's-complement representation of the number, the average value of the error has a bias of $2^{-b}/2$, whereas in all other cases just illustrated, the error has an average value of zero.

We shall use this statistical characterization of the quantization errors in our treatment of such errors in digital filtering and in the computation of the DFT for fixed-point implementation.

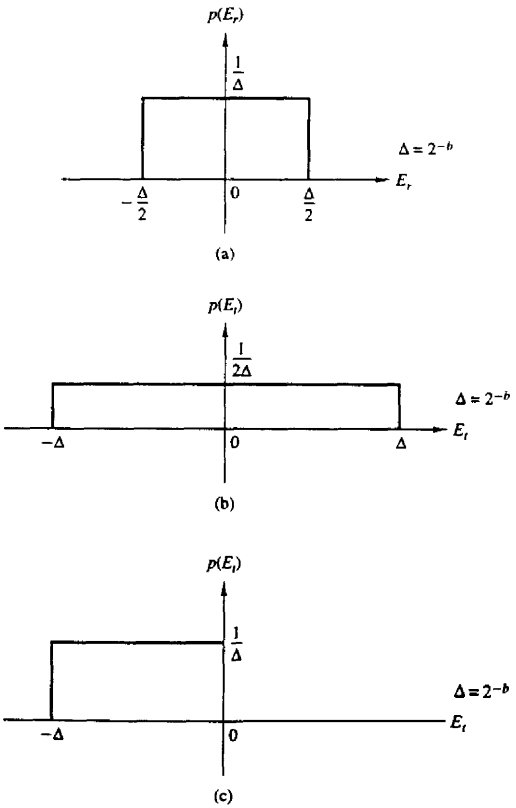


Figure 7.35 Statistical characterization of quantization errors: (a) round-off error, (b) truncation error for sign-magnitude; (c) truncation error for two's complement.

7.6 QUANTIZATION OF FILTER COEFFICIENTS

In the realization of FIR and IIR filters in hardware or in software on a general-purpose computer, the accuracy with which filter coefficients can be specified is limited by the word length of the computer or the length of the register provided to store the coefficients. Since the coefficients used in impiementing a given filter are not exact, the poles and zeros of the system function will, in general, be different from the desired poles and zeros. Consequently, we obtain a filter having a frequency response that is different from the frequency response of the filter with unquantized coefficients.

In Section 7.6.1, we demonstrate that the sensitivity of the filter frequency response characteristics to quantization of the filter coefficients is minimized by realizing a filter having a large number of poles and zeros as an interconnection of second-order fiher sections. This leads us to the parallel-form and cascade-form realizations in which the basic building blocks are second-order filter sections.

7.6.1 Analysis of Sensitivity to Quantization of Filter Coefficients

To illustrate the effect of quantization of the filter coefficients in a direct-form realization of an IIR filter, let us consider a general IIR fiher with system function

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}} \tag{7.6.1}$$

The direct-form realization of the IIR filter with quantized coefficients has the system function

$$\overline{H}(z) = \frac{\sum_{k=0}^{M} \overline{b}_k z^{-k}}{1 + \sum_{k=1}^{N} \overline{a}_k z^{-k}} \tag{7.6.2}$$

where the quantized coefficients $\{\overline{b}_k\}$ and $\{\overline{a}_k\}$ can be related to the unquantized coefficients $\{b_k\}$ and $\{a_k\}$ by the relations

$$\begin{aligned} \overline{a}_k &= a_k + \Delta a_k \quad k = 1, 2, \ldots, N \\ \overline{b}_k &= b_k + \Delta b_k \quad k = 0, 1, \ldots, M \end{aligned} \tag{7.6.3}$$

and $\{\Delta a_k\}$ and $\{\Delta b_k\}$ represent the quantization errors.

The denominator of $H(z)$ may be expressed in the form

$$D(z) = 1 + \sum_{k=0}^{N} a_k z^{-k} = \prod_{k=1}^{N} (1 - p_k z^{-1}) \tag{7.6.4}$$

where $\{p_k\}$ are the poles of $H(z)$. Similarly, we can express the denominator of $\overline{H}(z)$ as

$$\overline{D}(z) = \prod_{k=1}^{N} (1 - \overline{p}_k z^{-1}) \tag{7.6.5}$$

where $\overline{p}_k = p_k + \Delta p_k, k = 1, 2, \ldots, N$, and Δp_k is the error or perturbation resulting from the quantization of the filter coefficients.

We shall now relate the perturbation Δp_k to the quantization errors in the $\{a_k\}$.

The perturbation error Δp_i can be expressed as

$$\Delta p_i = \sum_{k=1}^{N} \frac{\partial p_i}{\partial a_k} \Delta a_k \tag{7.6.6}$$

where $\partial p_i / \partial a_k$, the partial derivative of p_i with respect to a_k, represents the incremental change in the pole p, due to a change in the coefficient a_k. Thus the total error Δp_i is expressed as a sum of the incremental errors due to changes in each of the coefficients $\{a_k\}$.

The partial derivatives $\partial p_i / \partial a_k$, k = 1, 2, ..., N, can be obtained by differentiating $D(z)$ with respect to each of the $\{a_k\}$. First we have

$$\left(\frac{\partial D(z)}{\partial a_k} \right)_{z=p_i} = \left(\frac{\partial D(z)}{\partial z} \right)_{z=p_i} \left(\frac{\partial p_i}{\partial a_k} \right) \tag{7.6.7}$$

Then

$$\frac{\partial p_i}{\partial a_k} = \frac{(\partial D(z)/\partial a_k)_{z=p_i}}{(\partial D(z)/\partial z)_{z=p_i}} \tag{7.6.8}$$

The numerator of (7.6.8) is

$$\left(\frac{\partial D(z)}{\partial a_k} \right)_{z=p_i} = -z^{-k}|_{z=p_i} = -p_i^{-k} \tag{7.6.9}$$

The denominator of (7.6.8) is

$$\left(\frac{\partial D(z)}{\partial z} \right)_{z=p_i} = \left\{ \frac{\partial}{\partial z} \left[\prod_{l=1}^{N} (1 - p_l z^{-1}) \right] \right\}_{z=p_i}$$

$$= \left\{ \sum_{k=1}^{N} \frac{p_k}{z^2} \prod_{\substack{l=1 \\ l \neq k}}^{N} (1 - p_l z^{-1}) \right\}_{z=p_i} \tag{7.6.10}$$

$$= \frac{1}{p_i^N} \prod_{\substack{l=1 \\ l \neq i}}^{N} (p_i - p_l)$$

Therefore, (7.6.8) can be expressed as

$$\frac{\partial p_i}{\partial a_k} = \frac{-p_i^{N-k}}{\prod_{\substack{l=1 \\ l \neq i}}^{N} (p_i - p_l)} \tag{7.6.11}$$

Substitution of the result in (7.6.11) into (7.6.6) yields the total perturbation error Δp_i in the form

$$\Delta p_i = -\sum_{k=1}^{N} \frac{p_i^{N-k}}{\prod_{\substack{l=1 \\ l \neq i}}^{N} (p_i - p_l)} \Delta a_k \tag{7.6.12}$$

This expression provides a measure of the sensitivity of the ith pole to changes in the coefficients $\{a_k\}$. An analogous result can be obtained for the sensitivity of the zeros to errors in the parameters $\{b_k\}$.

The terms $(p_i - p_l)$ in the denominator of (7.6.12) represent vectors in the z-plane from the poles $\{p_l\}$ to the pole p_i. If the poles are tightly clustered as they are in a narrowband filter, as illustrated in Fig. 7.36, the lengths $|p_i - p_l|$ are small for the poles in the vicinity of p_i. These small lengths will contribute to large errors and hence a large perturbation error Δp_i results.

The error Δp_i can be minimized by maximizing the lengths $|p_i - p_l|$. This can be accomplished by realizing the high-order filter with either single-pole or double-pole filter sections. In general, however, single-pole (and single-zero) filter sections have complex-valued poles and require complex-valued arithmetic operations for their realization. This problem can be avoided by combining complex-valued poles (and zeros) to form second-order filter sections. Since the complex-valued poles are usually sufficiently far apart, the perturbation errors $\{\Delta p_i\}$ are minimized. As a consequence, the resulting filter with quantized coefficients more closely approximates the frequency response characteristics of the filter with unquantized coefficients.

It is interesting to note that even in the case of a two-pole filter section, the structure used to realize the filter section plays an important role in the errors

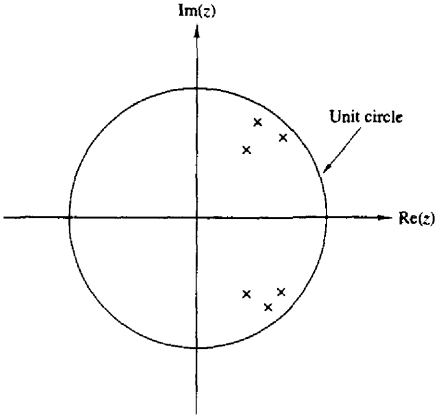


Figure 7.36 Pole positions for a bandpass IIR filter.

caused by coefficient quantization. To be specific, let us consider a two-pole filter with system function

$$H(z) = \frac{1}{1 - (2r\cos\theta)z^{-1} + r^2 z^{-2}} \tag{7.6.13}$$

This filter has poles at $z = re^{\pm j\theta}$. When realized as shown in Fig. 7.37, it has two coefficients, $a_1 = 2r\cos\theta$ and $a_2 = -r^2$. With infinite precision it is possible to achieve an infinite number of pole positions. Clearly, with finite precision (i.e., quantized coefficients a_1 and a_2), the possible pole positions are also finite. In fact, when b bits are used to represent the magnitudes of a_1 and a_2, there are at most $(2^b - 1)^2$ possible positions for the poles in each quandrant, excluding the case $a_1 = 0$ and $a_2 = 0$.

For example, suppose that $b = 4$. Then there are 15 possible nonzero values for a_1. There are also 15 possible values for r^2. We illustrate these possible values in Fig. 7.38 for the first quandrant of the z-plane only. There are 169 possible pole

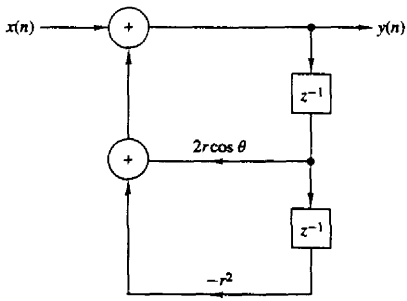


Figure 7.37 Realization of a two-pole IIR filter.

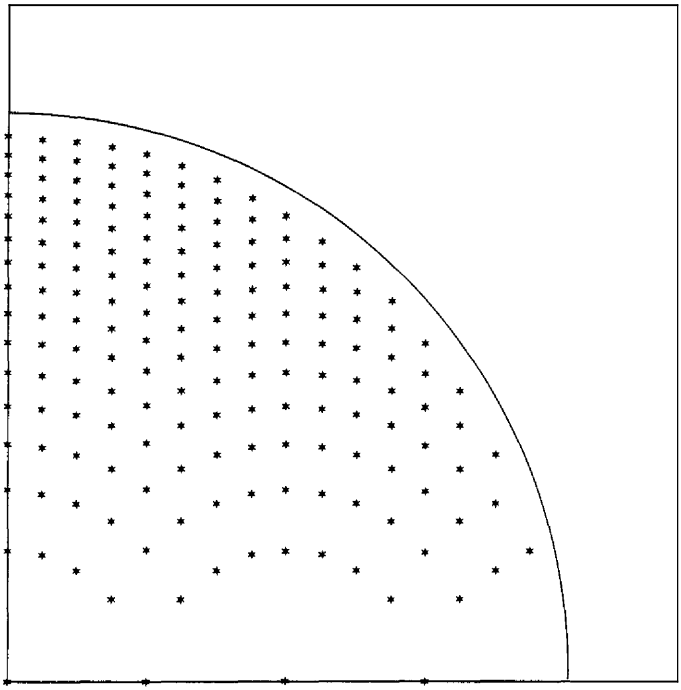


Figure 7.38 Possible pole positions for two-pole IIR filter realization in Fig. 7.37.

positions in this case. The nonuniformity in their positions is due to the fact that we are quantizing r^2, whereas the pole positions lie on a circular arc of radius r. Of particular significance is the sparse set of poles for values of θ near zero and, due to symmetry, near $\theta = \pi$. This situation would be highly unfavorable for lowpass filters and highpass filters which normally have poles clustered near $\theta = 0$ and $\theta = \pi$.

An alternative realization of the two-pole filter is the coupled-form realization illustrated in Fig. 7.39. The two coupled equations are

$$\begin{aligned} y_1(n) &= x(n) + r\cos\theta\ y_1(n-1) - r\sin\theta\ y(n-1) \\ y(n) &= r\sin\theta\ y_1(n-1) + r\cos\theta\ y(n-1) \end{aligned} \tag{7.6.14}$$

By transforming these two equations into the z-domain, it is a simple matter to show that

$$\frac{Y(z)}{X(z)} = H(z) = \frac{(r\sin\theta)z^{-1}}{1 - (2r\cos\theta)z^{-1} + r^2z^{-2}} \tag{7.6.15}$$

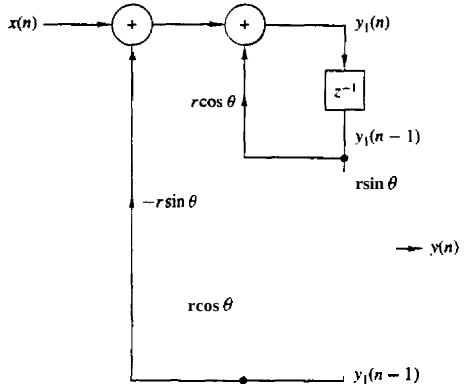


Figure 7.39 Coupled-form realization of a two-pole IIR filter.

In the coupled form we observe that there are also two coefficients, $\alpha_1 = r \sin\theta$ and $\alpha_2 = r\cos\theta$. Since they are both linear in r, the possible pole positions are now equally spaced points on a rectangular grid, as shown in Fig. 7.40, As a consequence, the pole positions are now uniformly distributed inside the unit circle, which is a more desirable situation than the previous realization, especially for lowpass filters. (There are 198 possible pole positions in this case.) However, the price that we pay for this uniform distribution of pole positions is an increase in computations. The coupled-form realization requires four multiplications per output point, whereas the realization in Fig. 7.37 requires only two multiplications per output point.

It is interesting to compare the coupled-form realization of Fig. 7.39 with the coupled (or normal) form state-space structure of Fig. 7.31. The poles of the state-space structure are directly related to its coefficients, since α_1 and α_2 are the real and imaginary parts of the roots. Since $\alpha_1 = r\cos\theta$ and $\alpha_2 = r\sin\theta$, it is clear that quantizing α_1 and α_2 results in a rectangular grid of possible pole positions, as shown in Fig. 7.40.

Since there are various ways in which one can realize a second-order filter section, there are obviously many possibilities for different pole locations with quantized coefficients. Ideally, we shouid select a structure that provides us with a dense set of points in the regions where the poles lie. Unfortunately, however, there is no simple and systematic method for determining the filter realization that yields this desired result.

Given that a higher-order IIR filter should be implemented as a combination of second-order sections, we still must decide whether to employ a parallel configuration or a cascade configuration. In other words, we must decide between the realization

$$H(z) = \prod_{k=1}^{K} \frac{b_{k0} + b_{k1}z^{-1} + b_{k2}z^{-2}}{1 + a_{k1}z^{-1} + a_{k2}z^{-2}} \tag{7.6.16}$$

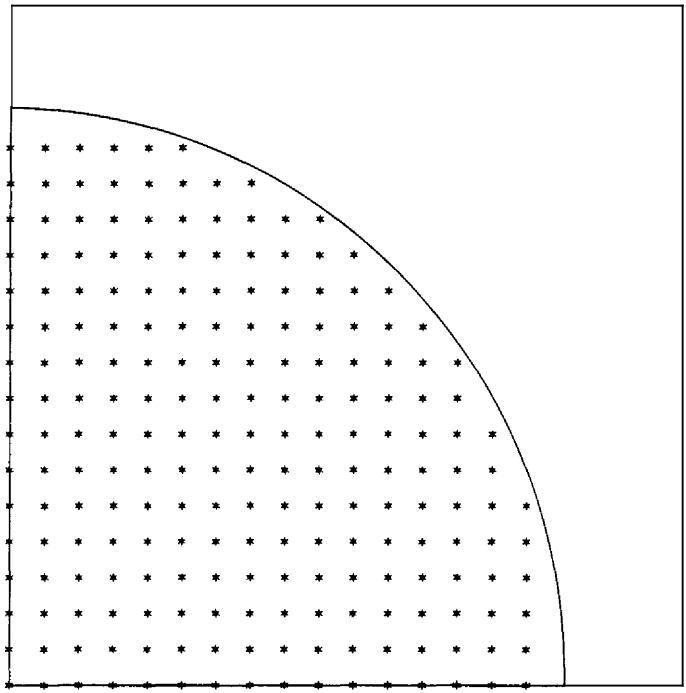


Figure 7.40 Possible pole positions for the coupled-form two-pole filter in Fig. 7.39.

and the realization

$$H(z) = \sum_{k=1}^{K} \frac{c_{k0} + c_{k1} z^{-1}}{1 + a_{k1} z^{-1} + a_{k2} z^{-2}} \tag{7.6.17}$$

If the IIR filter bas zeros on the unit circle, as is generally the case with elliptic and Chebyshev type II filters, each second-order section in the cascade configuration of (7.6.16) contains a pair of complex-conjugate zeros. The coefficients $\{b_k\}$ directly determine the location of these zeros. If the $\{b_k\}$ are quantized, the sensitivity of the system response to the quantization errors is easily and directly controlled by allocating a sufficiently large number of bits to the representation of the $\{b_{ki}\}$. In fact, we can easily evaluate the perturbation effect resulting from quantizing the coefficients $\{b_{ki}\}$ to some specified precision. Thus we have direct control of both the poles and the zeros that result from the quantization process.

On the other hand, the parallel realization of $H(z)$ provides direct control of the poles of the system only. The numerator coefficients $\{c_{k0}\}$ and $\{c_{k1}\}$ do not

specify the location of the zeros directly. In fact, the $\{c_{k0}\}$ and $\{c_{k1}\}$ are obtained by performing a partial-fraction expansion of $H(z)$. Hence they do not directly influence the location of the zeros, but only indirectly through a combination of all the factors of $H(z)$. As a consequence, it is more difficult to determine the effect of quantization errors in the coefficients $\{c_{ki}\}$, on the location of the zeros of the system.

It is apparent that quantization of the parameters $\{c_{ki}\}$ is likely to produce a significant perturbation of the zero positions and usually, it is sufficiently large in fixed-point implementations to move the zeros off the unit circle. This is a highly undesirable situation, which can be easily remedied by use of a floating-point representation. In any case the cascade form is more robust in the presence of coefficient quantization and should be the preferred choice in practical applications, especially where a fixed-point representation is employed.

Example 7.6.1

Determine the effect of parameter quantization on the frequency response of the 7-order elliptic filter given in Table 8.11 when it is realized as a cascade of second-order sections.

Solution The coefficients for the elliptic filter given in Table 8.11 are specified for the cascade form to six significant digits. We quantized these coefficients to four and then three significant digits (by rounding) and plotted the magnitude (in decibels) and the phase of the frequency response. The results are shown in Fig. 7.41 along the frequency response of the filter with unquantized (six significant digits) coefficients. We observe that there is an insignificant degradation due to coefficient quantization for the cascade realization.

Example 7.6.2

Repeat the computation of the frequency response for the elliptic filter considered in Example 7.6.1 when it is realized in the parallel form with second-order sections.

Solution The system function for the 7-order elliptic filter given in Table 8.11 is

$$H(z) = \frac{0.2781304 + 0.0054373108z^{-1}}{1 - 0.790103z^{-1}} + \frac{-0.3867805 + 0.3322229z^{-1}}{1 - 1.517223z^{-1} + 0.714088z^{-2}} + \frac{0.1277036 - 0.1558696z^{-1}}{1 - 1.421773z^{-1} + 0.861895z^{-2}} + \frac{-0.015824186 + 0.38377356z^{-1}}{1 - 1.387447z^{-1} + 0.962242z^{-2}}$$

The frequency response of this filter with coefficients quantized to four digits is shown in Fig. 7.42a. When this result is compared with the frequency response in Fig. 7.41, we observe that the zeros in the parallel realization have been perturbed sufficiently so that the nulls in the magnitude response are now at –80, –85, and –92 dB. The phase response has also been perturbed by a small amount.

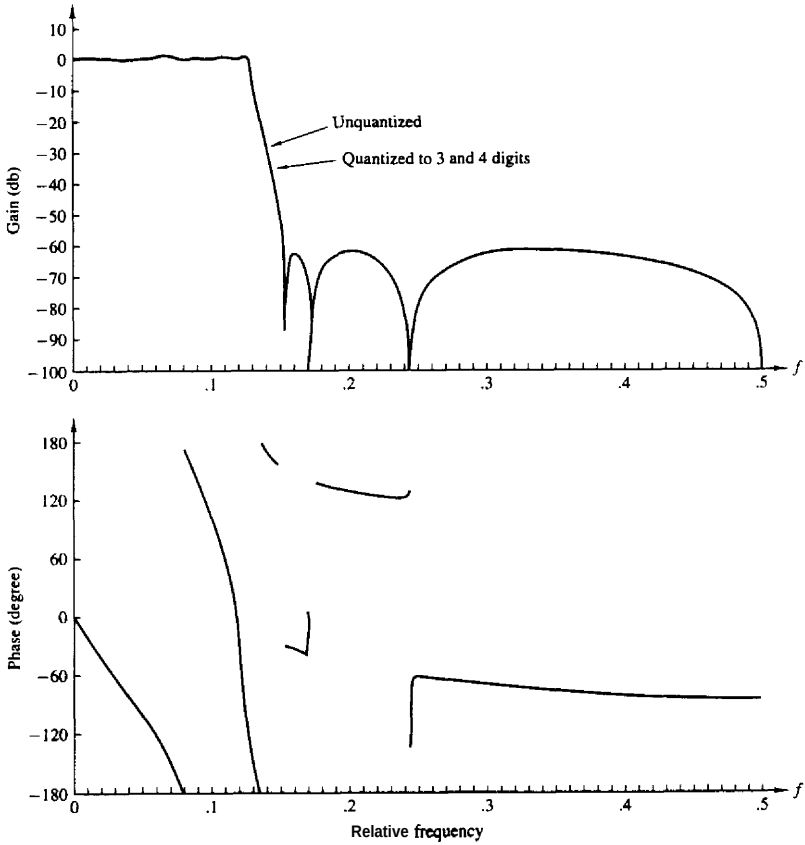


Figure 7.41 Effect of coefficient quantization of the magnitude and phase response of an $N = 7$ elliptic filter realized in cascade form.

When the coefficients are quantized to three significant digits, the frequency response characteristic deteriorates significantly, in both magnitude and phase, as illustrated in Fig. 7.42b. It is apparent from the magnitude response that the zeros are no longer on the unit circle as a result of the quantization of the coefficients. This result clearly illustrates the sensitivity of the zeros to quantization of the coefficients in the parallel form.

When compared with the results of Example 7.6.1, it is also apparent that the cascade form is definitely more robust to parameter quantization than the parallel form.

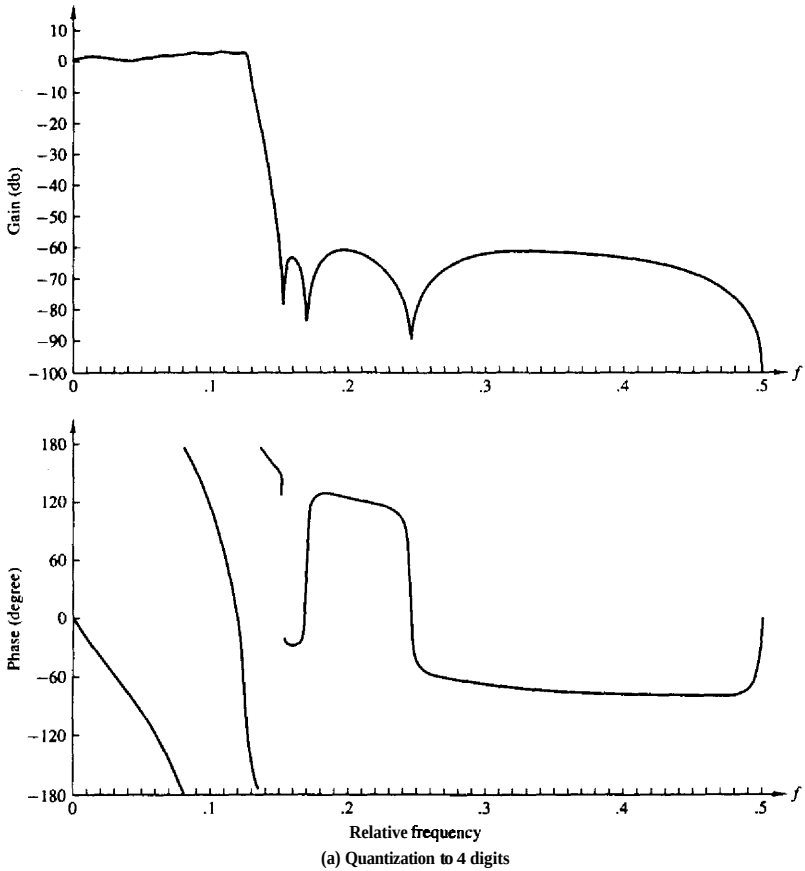
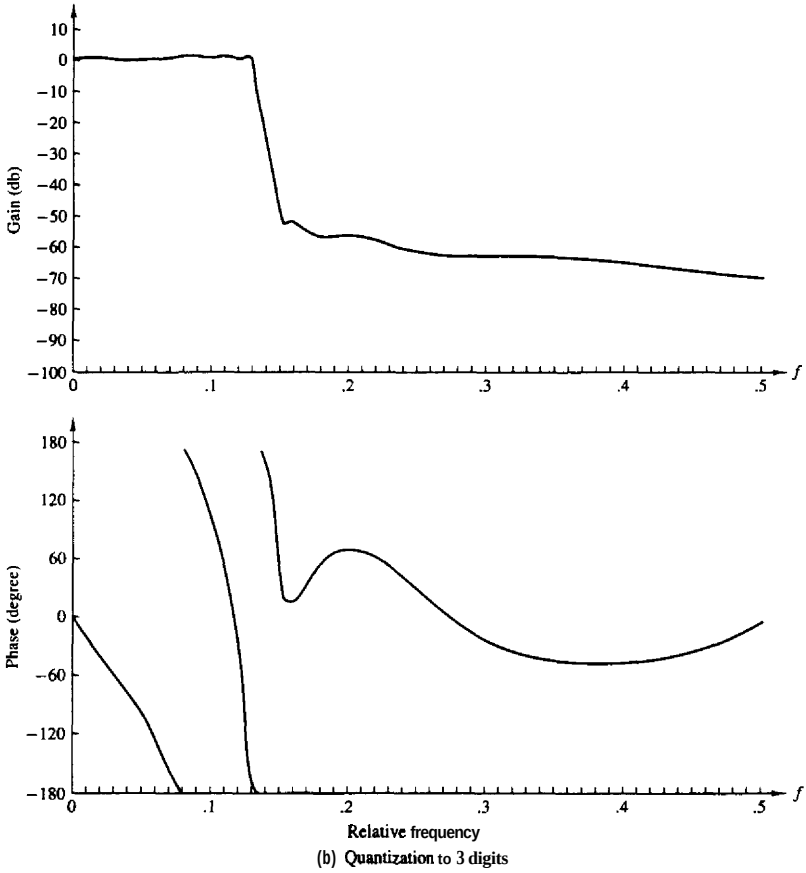


Figure 7.42 Effect of coefficient quantization of the magnitude and phase response of an $N = 7$ elliptic filter realized in cascade form: (a) quantization to four digits; (b) quantization to three digits.

7.6.2 Quantization of Coefficients in FIR Filters

As indicated in the preceding section, the sensitivity analysis performed on the poles of a system also applies directly to the zeros of the IIR filters. Consequently, an expression analogous to (7.6.12) can be obtained for the zeros of an FIR filter. In effect, we should generally realize FIR filters with a large number of zeros as



(b) Quantization to 3 digits

Figure 7.42 *Continued*

a cascade of second-order and first-order filter sections to minimize the sensitivity to coefficient quantization.

Of particular interest in practice is the realization of linear phase FIR filters. The direct-form realizations shown in Figs. 7.1 and 7.2 maintain the linear-phase property even when the coefficients are quantized. This follows easily from the observation that the system function of a linear-phase FIR filter satisfies the property

$$H(z) = \pm z^{-(M-1)} H(z^{-1})$$

independent of whether the coefficients are quantized or unquantized (see Section 8.2). Consequently, coefficient quantization does not affect the phase characteristic of the FIR filter, but affects only the magnitude. As a result, coefficient quantization effects are not as severe on a linear-phase FIR filter, since the only effect is in the magnitude.

Example 7.63

Determine the effect of parameter quantization on the frequency response of an $M = 32$ linear-phase FIR bandpass filter. The filter is realized in the direct form.

Solution The frequency response of a linear-phase FIR bandpass filter with unquantized coefficients is illustrated in Fig. 7.43a. When the coefficients are quantized to four significant digits, the effect on the frequency response is insignificant. However, when the coefficients are quantized to three significant digits, the sidelobes increased by several decibels, as illustrated in Fig. 7.43b. This result indicates that we should use a minimum of 10 bits to represent the coefficients of this FIR filter and, preferably, 12 to 14 bits, if possible.

From this example we learn that a minimum of 10 bits is required to represent the coefficients in a direct-form realization of an FIR filter of moderate length. As the filter length increases, the number of bits per coefficient must be increased to maintain the same error in the frequency response characteristic of the filter.

For example, suppose that each filter coefficient is rounded to $(b + 1)$ bits. Then the maximum error in a coefficient value is bounded as

$$-2^{-(b+1)} < e_h(n) < 2^{-(b+1)}$$

Since the quantized values may be represented as $\bar{h}(n) = h(n) + e_h(n)$, the error in the frequency response is

$$E_M(\omega) = \sum_{n=0}^{M-1} e_h(n) e^{-j\omega n}$$

Since $e_h(n)$ is zero mean, it follows that $E_M(\omega)$ is also zero mean. Assuming that the coefficient error sequence $e_h(n)$, $0 \le n \le M-1$, is uncorrelated, the variance of the error $E_M(\omega)$ in the frequency response is just the sum of the variances of the M terms. Thus we have

$$\sigma_E^2 = \frac{2^{-2(b+1)}}{12} M = \frac{2^{-2(b+2)}}{3} M$$

Here we note that the variance of the error in $H(\omega)$ increases linearly with M. Hence the standard deviation of the error in $H(\omega)$ is

$$\sigma_E = \frac{2^{-(b+2)}}{\sqrt{3}} \sqrt{M}$$

Consequently, for every factor of 4 increase in M, the precision in the filter coefficients must be increased by one additional bit to maintain the standard deviation fixed. This result, taken together with the results of Example 7.6.3, implies that

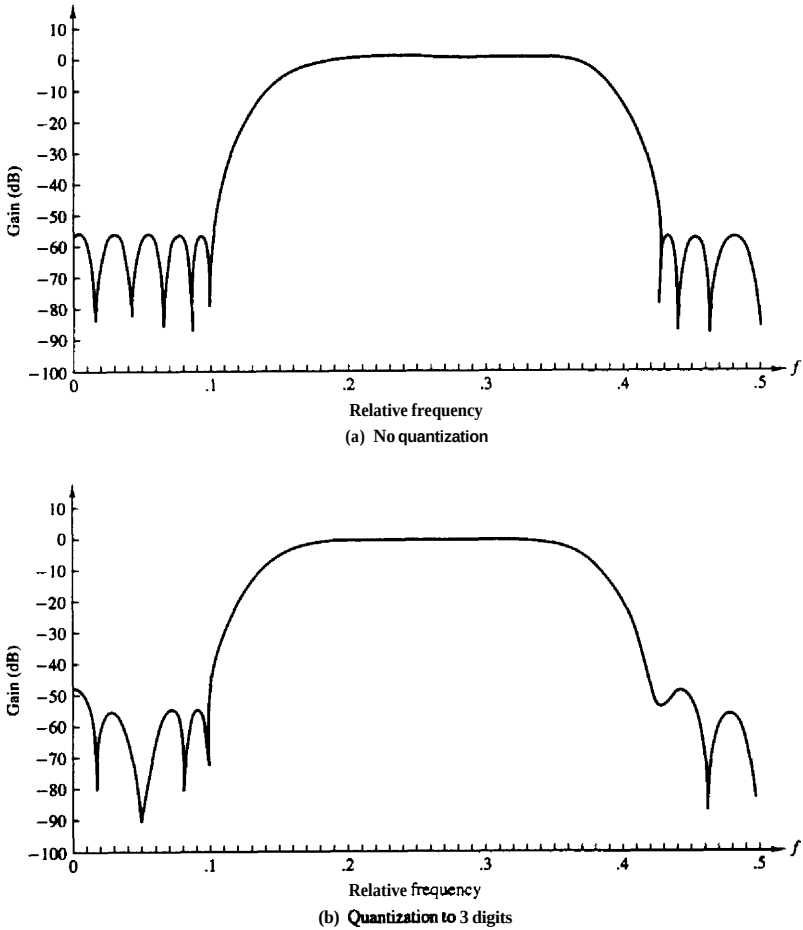


Figure 7.43 Effect of coefficient quantization of the magnitude of an $M = 32$ linear-phase FIR filter realized in direct form: (a) no quantization; (b) quantization to three digits.

the frequency error remains tolerable for filter lengths up to 256, provided that filter coefficients are represented by 12 to 13 bits. If the word length of the digital signal processor is less than 12 bits or if the filter length exceeds 256, the filter should *be* implemented as a cascade of smaller length filters to reduce the precision requirements.

In a cascade realization of the form

$$H(z) = G \prod_{k=1}^{K} H_k(z) \tag{7.6.18}$$

where the second-order sections are given as

$$H_k(z) = 1 + b_{k1}z^{-1} + b_{k2}z^{-2} \tag{7.6.19}$$

the coefficients of complex-valued zeros are expressed as $b_{k1} = -2r_k \cos\theta_k$ and $b_{k2} = r_k^2$. Quantization of b_{k1} and b_{k2} result in zero locations as shown in Fig. 7.38, except that the grid extends to points outside the unit circle.

A problem may arise, in this case, in maintaining the linear-phase property, because the quantized pair of zeros at $z = (1/r_k)e^{\pm j\theta_k}$ may not be the mirror image of the quantized zeros at $z = r_k e^{\pm j\theta_k}$. This problem can be avoided by rearranging the factors corresponding to the mirror-image zero. That is, we can write the mirror-image factor as

$$\left(1 - \frac{2}{r_k}\cos\theta_k z^{-1} + \frac{1}{r_k^2}z^{-2}\right) = \frac{1}{r_k^2}(r_k^2 - 2r_k \cos\theta_k z^{-1} + z^{-2}) \tag{7.6.20}$$

The factors $\{1/r_k^2\}$ can be combined with the overall gain factor G, or they can be distributed in each of the second-order filters. The factor in (7.6.20) contains exactly the same parameters as the factor $(1 - 2r_k \cos\theta_k z^{-1} + r_k^2 z^{-2})$, and consequently, the zeros now occur in mirror-image pairs even when the parameters are quantized.

In this brief treatment we have given the reader an introduction to the problems of coefficient quantization in IIR and FIR filters. We have demonstated that a high-order filter should be reduced to a cascade (for FIR or IIR filters) or a parallel (for IER filters) realization to minimize the effects of quantization errors in the coefficients. This is especially important in fixed-point realizations in which the coefficients are represented by a relatively small number of bits.

7.7 ROUND-OFF EFFECTS IN DIGITAL FILTERS

In Section 7.5 we characterized the quantization errors that occur in arithmetic operations performed in a digital filter. The presence of one or more quantizers in the realization of a digital filter results in a nonlinear device with characteristics that may be significantly different from the ideal linear filter. For example, a recursive digital filter may exhibit undesirable oscillations in its output, as shown in the following section, even in the absence of an input signal.

As a result of the finite-precision arithmetic operations performed in the digital filter, some registers may overflow if the input signal level becomes large.

Overflow represents another form of undesirable nonlinear distortion on the desired signal at the output of the filter. Consequently, special care must be exercised to scale the input signal properly, either to prevent overflow completely or, at least, to minimize its rate of occurrence.

The nonlinear effects due to finite-precision arithmetic make it extremely difficult to precisely analyze the performance of a digital filter. To perform an analysis of quantization effects, we adopt a statistical characterization of quantization errors which, in effect, results in a linear model for the filter. Thus we are able to quantify the effects of quantization errors in the implementation of digital filters. Our treatment is limited to fixed-point realizations where quantization effects are very important.

7.7.1 Limit-Cycle Oscillations in Recursive Systems

In the realization of a digital filter, either in digital hardware or in software on a digital computer, the quantization inherent in the finite- precision arithmetic operations render the system nonlinear. In recursive systems. the nonlinearities due to the finite-precision arithmetic operations often cause periodic oscillations to occur in the output, even when the input sequence is zero or some nonzero constant value. Such oscillations in recursive systems are called *limit* cycles and are directly attributable to round-off errors in multiplication and overflow errors in addition.

To illustrate the characteristics of a limit-cycle oscillation, let us consider a single-pole system described by the linear difference equation

$$y(n) = ay(n-1) + x(n) \tag{7.7.1}$$

where the pole is at $z = a$. The ideal system is realized as shown in Fig. 7.44. On the other hand, the actual system, which is described by the nonlinear difference equation

$$v(n) = Q[av(n-1)] + x(n) \tag{7.7.2}$$

is realized as shown in Fig. 7.45.

Suppose that the actual system in Fig. 7.45 is implemented with fixed-point arithmetic based on four bits for the magnitude plus a sign bit. The quantization that takes place after multiplication is assumed to round the resulting product upward.

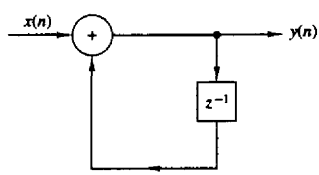


Figure 7.44 Ideal single-pole recursive system.